# Project Report – StudyHelper

**Institution:**
VIA University College

**Course & Semester:**
SEP4, 4th Semester

**Authors (Student Numbers):**
Alexandru Savin(354790)
Cristina Aurelia Matei (354776)
Damian Michal Choina (354789)
Eduard Fekete (355323)
Jakub Maciej Baczek (354814)
Karina Rubahova (354565)
Mara-Ioana Statie (354536)
Marta Zrno (355351)
Piotr Junosz (355502)
Tymoteusz Krzysztof Zydkiewicz (355413)

**Supervisors:**
Erland Ketil Larsen
Joseph Chukwudi Okika
Kasper Knop Rasmussen
Markéta Tranberg

**Date:**
May 25, 2026

**Character Count:**
118,401 characters

# TABLE OF CONTENTS

# ABSTRACT

Authors: [Name, Name, Name]

[Concise summary of the entire project. State the problem being solved, the three-part technical approach (embedded IoT on ATmega2560, machine learning pipeline, React web frontend), the cloud infrastructure, and the main results. Should stand alone as a paragraph that lets a reader decide whether to read further.]

[TODO: each team check and adjust sentences about their parts of the system so its 100% correct]

StudyHelper is a distributed study-environment monitoring system developed to address the challenge of suboptimal indoor conditions affecting student concentration and academic performance. The system integrates three tightly coupled components: an embedded IoT device based on the ATmega2560 microcontroller that measures temperature, humidity, $CO_2$ concentration, and light; a machine learning pipeline implemented in Python that predicts a Study Suitability Rating from aggregated sensor data and a React web frontend that presents live readings, historical sensor readings, and ML-generated predictions to the user. All components are deployed as Docker containers on a cloud host managed through Coolify, with the Core API (FastAPI) acting as the central gateway - persisting sensor data in a shared PostgreSQL database and forwarding requests to the MAL FastAPI service for suitability predictions. The IoT firmware communicates with the backend over HTTP using a session-based protocol, transmitting sensor payloads every 30 seconds and keepalive pulses every five seconds. The machine learning model [TODO: update after making the final model] - a Random Forest classifier trained on environmental datasets with MICE imputation - produces both instant and trend based sustainability ratings on a scale of one to five, which are shown to users through the web interface. [Summarise the main quantitative results here, e.g. model accuracy, test coverage, and system uptime, once final figures are available.]

# 1. INTRODUCTION

Authors: [Name, Name, Name]

## 1.1 Background and Motivation

[Describe the domain context. What real-world problem does the system address? Why is an IoT-based solution with machine learning relevant here? Who are the stakeholders and what do they need?]

The physical environment in which learning takes place has a measurable impact on cognitive performance. Research has shown that environmental conditions such as temperature, humidity, $CO_2$ concentration, and lighting directly influence students' ability to concentrate and retain information (Bustamante-Mora et al., 2025) [TODO: add reference in references]. Despite growing awareness of this relationship, most study spaces - libraries, classrooms, dormitories - provide no real-time feedback on whether the ambient conditions are conducive to productive work. Students are left to rely on subjective perception, often noticing that conditions are poor only after their focus has already deteriorated. This makes the problem difficult to handle manually, because students may feel tired or distracted without knowing whether the cause is related to the environment. Objective measurements can help make these conditions visible before they noticeably affect the study session.

The problem is particularly acute in shared environments where individual control over heating, ventilation, or lighting is limited or absent. A student in a crowded library has no practical way of knowing whether the rising $CO_2$ level in the room is contributing to their fatigue, or whether the ambient temperature is outside the range associated with optimal cognitive function. Existing commercial solutions either focus on a single sensor parameter or require expensive infrastructure that is not feasible in a typical university context. For schools and universities, this creates a practical need for a solution that can monitor indoor conditions without requiring expensive building-wide infrastructure.

By combining low-cost IoT sensing with a machine learning prediction layer, the StudyHelper system closes the gap between raw environmental data and actionable guidance. Rather than simply logging sensor values, the system learns from historical patterns of user-rated study sessions and uses that knowledge to predict how suitable current conditions are likely to be - providing students and teachers with a tool to make informed decisions about when and where to study.

The primary stakeholders are students, who benefit directly from improved self-awareness of their study environment; teachers, who can use condition overviews to assess classroom suitability. [Expand here if additional stakeholder interviews or literature sources are added.]

## 1.2 Problem Statement

[State the specific problem the project addresses. Be precise about what is currently missing or suboptimal, and what a successful solution would look like.]

Based on the environmental challenges identified in the problem domain, this project addresses the following core question:

**Main problem:** How can indoor environmental data be monitored and analyzed to better understand how environmental conditions affect the quality of students' study sessions?

This problem is decomposed into the following sub-questions:

- What are the most significant indoor environmental factors that affect students' study conditions?
- What factors can be used to represent study session quality and indoor environmental quality?
- How do students react to changes in indoor environmental quality?
- Is there a measurable relationship between environmental factors and user-provided study ratings?
- How accurately can study suitability be predicted based on collected sensor data?

A successful solution would continuously collect real-world sensor data from study environments, expose this data through a structured API, and deliver ML-driven suitability predictions to students via a responsive web interface. The system should require minimal manual input from the user, apart from connecting a device and optionally submitting a post-session quality rating.

## 1.3 Project Objectives

[Enumerate concrete, verifiable goals. These will be evaluated against in Results and Discussion. Tie them to the three system components where relevant.

The concrete objectives of the StudyHelper project are as follows:

- **IoT:** The embedded device shall measure temperature, humidity, $CO_2$ concentration, and light level using the ATmega2560 MCU and transmit structured JSON payloads to the cloud backend via HTTP at a regular interval not exceeding sixty seconds.
- **Cloud Backend:** The Core API (FastAPI) shall act as the central gateway - persisting sensor readings and session metadata in a shared PostgreSQL database and exposing a RESTful API consumed by both the frontend and the MAL service.
- **Machine Learning:** The ML component shall train a classifier capable of predicting a Study Suitability Rating on an integer scale of one to five, using aggregated temperature window features derived from historical sensor data, and expose predictions through a FastAPI endpoint.
- **Frontend:** The React web application shall display live sensor readings and the current ML-predicted suitability rating in a responsive layout that adapts to screen widths of 576 px, 768 px, and 1200 px.
- **DevOps:** All components shall be containerised with Docker, deployed to a public cloud host, and supported by automated CI/CD pipelines on GitHub Actions that enforce unit-test passing and successful build compilation before merging to the main branch.
- **Security:** Communication between the IoT device and the backend shall be protected by [encryption scheme to be specified]; frontend-facing API endpoints shall be protected by [JWT / API key to be confirmed].

[TODO: Review these objectives against the final delivered system before submission and adjust the wording where the implementation diverged from the original plan.]

## 1.4 Scope and Delimitations

[Define the boundaries. What is explicitly in scope for each component? What has been consciously left out, and why? Important given the 60-page limit.]

The system is designed around a physical IoT device that can be connected to a user account and used in different study environments, such as classrooms, libraries, or personal study spaces. The following delimitations were agreed at project inception and are reflected in the implemented system:

**Geographical scope:** The device is designed and tested within a campus-related study context, such as classrooms, libraries, or personal study spaces. The IoT device is treated as a portable physical sensor that can be connected to a user account and used in the environment where the user is currently studying. Multi-building deployment is out of scope for this iteration.

**Sensor coverage:** The system measures temperature, humidity, $CO_2$ concentration, and light. Sound level measurement is not currently implemented by the active firmware, despite being noted as a candidate sensor in the project description due to the malfunctioning and inaccurate quality of the sound sensor.

**Automation:** The system provides predictive guidance but does not automate any physical actuators beyond the onboard buzzer alert triggered when the predicted study quality rating reaches the lowest level. Full HVAC or lighting automation is explicitly out of scope.

**User model:** The system supports user roles such as student who can start and stop sessions via the physical button on the device and submit a post-session quality rating through the frontend, and a teacher who assesses whether conditions are suitable for teaching. Administrator views are out of scope.

**Privacy and data collection:** The system stores environmental sensor readings and session lifecycle metadata. In addition, user accounts are stored, including name, last name, email address, and a hashed password, as well as optional profile fields such as university, study programme, study year, study goal, and a profile picture reference. Connected device information and post-session ratings submitted by users are persisted so ratings can be associated with the correct device and study session. No audio recordings or video are collected. All personally identifiable data is linked to a user account and is subject to standard data protection considerations; password storage uses hashing.

**Medical and psychological analysis:** The system does not attempt to measure or infer physiological state, stress levels, or cognitive load directly. The Study Suitability Rating is an environmental proxy, not a clinical assessment.

[Add any additional delimitations agreed with supervisors, particularly regarding scale of the ML dataset or the number of test participants.]

## 1.5 Related Work

[Survey existing IoT, ML, and web solutions relevant to your domain. Cite with APA. How does your system differ from or build on existing approaches?]

A growing body of research has examined the relationship between indoor environmental quality (IEQ) and cognitive performance. Bustamante-Mora et al. (2025)[TODO: add reference] conducted a case study in university environments demonstrating that temperature and $CO_2$ levels correlate with measurable changes in student concentration and learning outcomes. Their findings motivate the sensor selection in this project, specifically the inclusion of $CO_2$ as a key environmental indicator alongside temperature and humidity.

IoT-based classroom monitoring systems have been explored in several contexts. [Cite one or two relevant prior IoT classroom studies here, e.g. systems based on Raspberry Pi or Arduino platforms.] These systems typically focus on data logging and threshold-based alerting rather than predictive modelling, which is the distinguishing contribution of the StudyHelper approach.

In the domain of machine learning for indoor environment prediction, [cite relevant ML papers on predicting thermal comfort or cognitive performance from environmental data]. The use of ensemble methods such as Random Forest[TODO: update after final model choice] for environmental classification has been validated in comparable contexts and supports the model choice made in this project.

On the frontend side, React-based dashboards for IoT data visualisation are a well-established pattern. [Reference any relevant existing open-source dashboards or comparable student-facing tools.]

StudyHelper differs from prior work in its end-to-end integration: rather than treating sensing, prediction, and presentation as separate research artefacts, the project combines all three within a single deployable system. The use of a user-provided post-session rating as the supervised learning target - rather than a physiologically measured focus indicator - is a pragmatic design choice that makes the system trainable without invasive data collection, and represents a novel framing of the suitability prediction problem.

[This section should be expanded with full APA citations before submission. The references listed here are indicative placeholders.]

# 2. ANALYSIS

Authors: [Name, Name, Name]

# 2.1 Domain Model

The domain model represents the structural backbone of the StudyHelper platform, mapping physical hardware contexts to virtual user spaces and persistent storage entities. By conceptualizing the domain prior to database modeling, the system achieves strict normalization and clear separation of concerns.

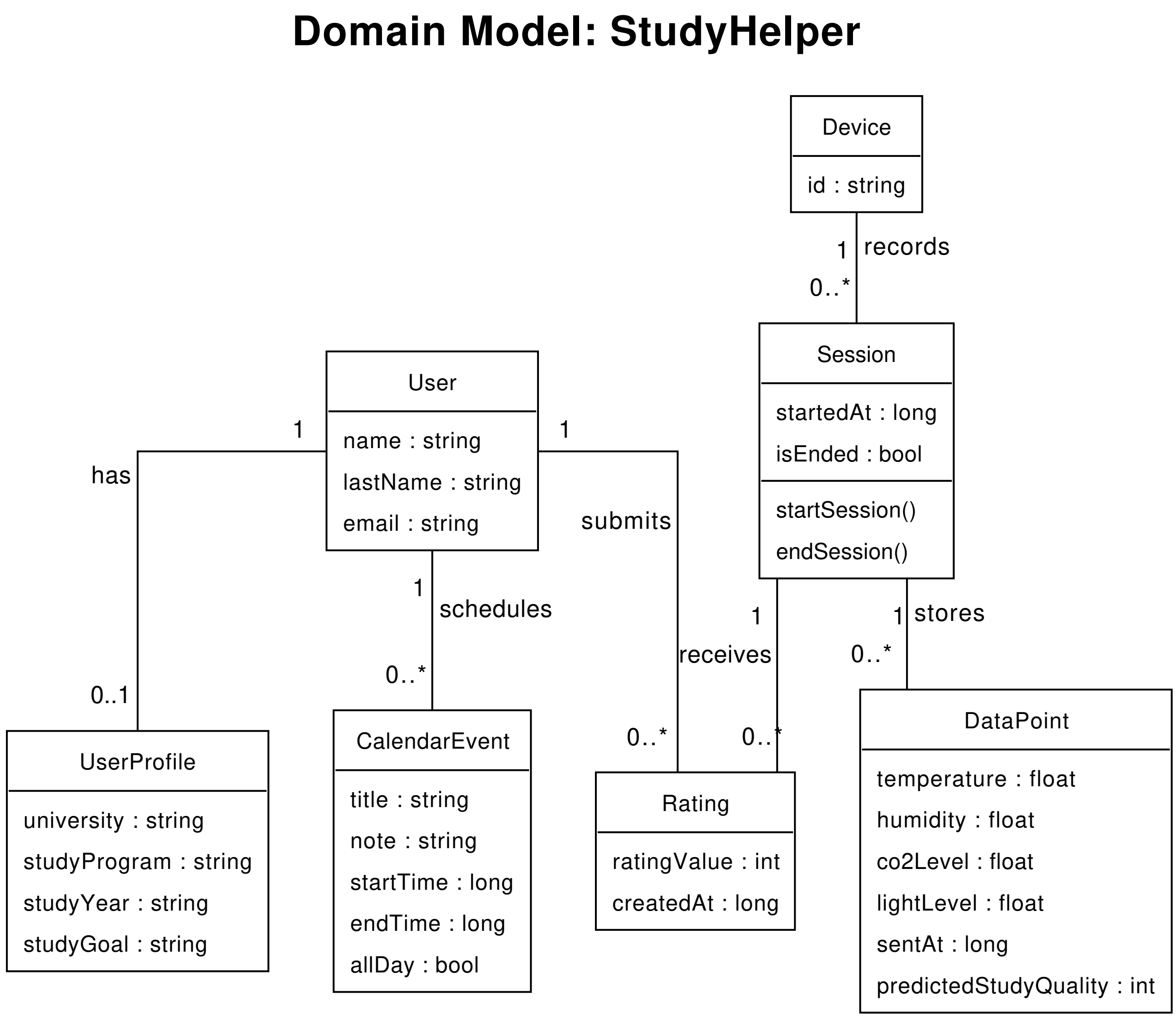



**Figure 1:** Domain Model

Figure 2.1: Conceptual Domain Model of the StudyHelper System.

## Architectural and Structural Breakdown

The domain is partitioned into distinct logical spaces that coordinate during active monitoring:

- **Identity and Personalization Space (`User` & `UserProfile`)**: In typical authentication designs, user credentials and preferences are coupled. In StudyHelper, the `User` entity (retaining name, email, and authentication hashes) is structurally decoupled from the `UserProfile` via a 1-to-0..1 relationship. This separation allows the authentication engine to remain lightweight and performant. The `UserProfile` captures academic metadata (`university`, `studyProgram`, `studyYear`, `studyGoal`, `profilePicture`). Although the database schema (`initdb/01_schema.sql`) and Core REST API models contain preferred comfort targets (`preferred_temp` and `preferred_co2`), these are excluded from the active system boundary domain model in this iteration because the client application does not save or display them.
- **Physical Device Space (`Device`)**: A `Device` represents a physical hardware unit deployed in a student's room, uniquely identified by a hardware-defined string `id`. In the relational model, a `Device` exists independently of a `User`, supporting a multi-tenant device lifecycle where a device can be reassigned or shared.
- **Monitoring Event Space (`Session`)**: Rather than registering telemetry as a continuous uncontextualized stream, the domain groups data points into discrete `Session` blocks. A `Session` captures the duration of a specific study period, initialized and finalized via physical buttons on the device. It maintains a state flag `isEnded` and records the lifecycle via `startedAt` and `lastPulseAt` timestamps.
- **Telemetry Space (`DataPoint`)**: Representing the actual physical measurements, a `DataPoint` encapsulates temperature, humidity, $CO_2$ level, and light level. Each data point is strictly coupled to a parent `Session` (1-to-many relationship). This design choice is critical for the predictive model, as it associates sensor snapshots with the specific temporal window of a study session. Additionally, each data point stores the `predictedStudyQuality` generated by the machine learning pipeline at the moment of ingestion.
- **Feedback Space (`Rating`)**: A `Rating` provides the user's subjective assessment of their comfort and concentration level during a session on a scale of 1 to 5 (`ratingValue`). While the underlying database schema and Core REST API support a nullable `comment` string, it is excluded from the active use cases and sequence diagrams in this release due to the absence of input fields in the user interface application.
- **Planning Space (`CalendarEvent`)**: Enables students to schedule future study blocks directly in the app. Each `CalendarEvent` is owned by a `User` and contains scheduling parameters (`title`, `note`, `startTime`, `endTime`, `allDay`).

### Analysis of Conceptual Modeling Decisions

1. **Session-Telemetry Coupling for Feature Extraction**: Attaching `DataPoints` to a `Session` rather than directly to a `Device` or `User` provides a natural grouping for statistical aggregation. The Machine Learning service (MAL) requires session-wide aggregates (such as mean temperature, $CO_2$ variance, and light exposure duration) to formulate accurate comfort predictions. This structural coupling avoids complex windowing queries in PostgreSQL during inference.
2. **Postponed Preference and Comment Integration**: To streamline frontend development in this release, comfort target preferences (`preferred_temp`/`preferred_co2` in `UserProfile`) and qualitative comments (in `Rating`) are not exposed to the user interface. However, the database schema (`initdb/01_schema.sql`) and Core REST API endpoints preserve these attributes. This intentional design choice ensures backend-to-frontend compatibility for future releases without requiring schema migrations or endpoint redesigns.
3. **Decoupled User Profiles for Compliance and Agility**: Decoupling `UserProfile` from `User` ensures that personal data storage is separated. If a user requests account deletion under data privacy regulations, the `UserProfile` and `User` tables can be purged without breaking the anonymized telemetry database, since the historical `Session` data is linked only via the device identifier.

## 2.2 User Stories

The following user stories were derived from the use cases defined in the project analysis and reflect the agreed shared backlog across all three sub-teams:

| ID | User Story | Priority | Component |
|---|---|---|---|
| US01 | As a Student, I want an overview of current environmental conditions and a suitability prediction, so that I can decide whether to start or continue a study session. | Must have | IoT / Cloud / ML / Frontend |
| US02 | As a Student, I want to submit a rating after a study session, so that the system can improve suitability predictions over time. | Must have | Frontend / ML |
| US03 | | | |

| ID | User Story | Priority | Component |
|---|---|---|---|
| | As a Student, I want to see the history of environmental conditions during my study session, so that I can understand trends affecting my focus. | Should have | Frontend / Cloud |
| US04 | As a Student, I want to receive an alert when conditions fall below an acceptable threshold, so that I can take action to improve my study environment. | Should have | IoT / Cloud |
| US05 | As a Student, I want to start and stop a monitored study session using a physical button on the device, so that I do not need to interact with any software to begin monitoring. | Must have | IoT |
| US06 | As a Teacher, I want an overview of current and predicted environmental conditions, so that I can decide if the environment is suitable for teaching. | Could have | Frontend / ML |
| US07 | As a Student, I want the web application to work correctly on my phone, tablet, and laptop, so that I can check conditions from any device. | Should have | Frontend |
| US08 | As a User, I want to register, log in, and log out securely, so that my personal data and study information are protected. | Must have | Frontend |
| US09 | As a Student, I want to connect my physical device to my account, so that the dashboard can show data from my own study device. | Must have | Frontend |
| US10 | As a Student, I want to manage my profile information, so that the application can be personalized to me. | Should have | Frontend |
| US11 | As a Student, I want to manage calendar events, so that I can plan study sessions in the same application. | Could have | Frontend |
| US12 | As a User, I want to switch between English and Danish and use dark mode, so that the application is easier to use in different contexts. | Could have | Frontend |

## 2.3 System Requirements

The following requirements were derived from the user stories and supplemented by cross-cutting concerns identified during analysis:

| ID | Requirement Description | Type | Source |
|---|---|---|---|
| FR01 | The IoT device shall measure temperature (°C), humidity (%), $CO_2$ (ppm), and light level (raw ADC), and transmit a JSON payload to the backend API at intervals not exceeding 60 seconds during an active session. | Functional | US01, US05 |
| FR02 | The backend shall persist sensor readings, device identities, and session records in a PostgreSQL database. | Functional | US01, US03 |
| FR03 | The backend API shall expose endpoints for device registration (`POST /device`), session management (`POST /session`, `PATCH /session/{id}/pulse`), and data submission (`POST /data`). | Functional | US01, US05 |
| FR04 | The ML service shall expose a `POST /predict` endpoint returning an integer Study Suitability Rating between 1 and 5. | Functional | US01, US06 |
| FR05 | The frontend shall display the current sensor readings, the ML-predicted suitability rating, and a time-series chart of historical readings for the active session. | Functional | US01, US03 |
| FR06 | The frontend shall allow a student to submit a post-session quality rating, which is stored against the session record. | Functional | US02 |
| FR07 | The IoT device shall sound the onboard buzzer when the predicted study quality returned by the server reaches level 1 (lowest). | Functional | US04 |
| NFR01 | The user interface must adapt dynamically without horizontal scrolling or text clipping across screen widths from 320 px320\text{ px} to 1920 px1920\text{ px}. | Non-functional | US07 |
| NFR02 | | | Reliability |

| ID | Requirement Description | Type | Source |
|---|---|---|---|
| | The backend must automatically end a study session if no keepalive signal is received from the device for a continuous period of 30 seconds30\text{ seconds}. | Non-functional | |
| NFR03 | The ML prediction service must return comfort ratings in less than 200 ms200\text{ ms} for 99%99\% of requests under concurrent load. | Non-functional | US01, US06 |
| NFR04 | All server-side applications must be deployable to a target environment using a single orchestration command in under 5 minutes5\text{ minutes}. | Non-functional | Supportability |
| NFR05 | The authentication service must temporarily lock a user account for 15 minutes15\text{ minutes} after 5 consecutive failed login attempts. | Non-functional | Security |
| NFR06 | The CI/CD pipeline must execute automated unit tests and block code merges if test coverage falls below 80%80\% or if any test fails. | Non-functional | DevOps |

A comprehensive specification of these non-functional criteria under the FURPS+ quality framework is maintained in the project's Non-Functional Requirements Document.

# 2.4 System Sequence Diagrams

System Sequence Diagrams (SSDs) establish the boundaries of the StudyHelper system, mapping external actor triggers to synchronous and asynchronous system responses. In our architectural design, these SSDs served as the contract between the embedded firmware (C/C++ on ATmega2560), the cloud backend (FastAPI/PostgreSQL), the machine learning service (MAL), and the client-side user interface (React).

## 2.4.1 IoT-Initiated Session Lifecycles

The start and end use cases governed by the physical button interface on the hardware box require precise synchronization between the microcontroller's local state and the database state.

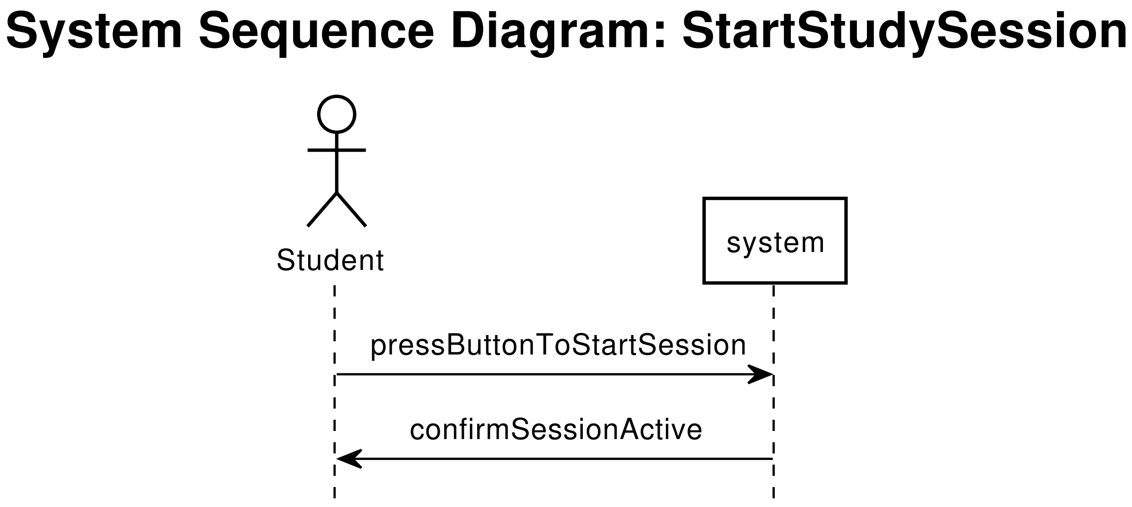



**Figure 2:** SSD: Start Study Session

Figure 2.2: System Sequence Diagram for IoT-initiated session startup.

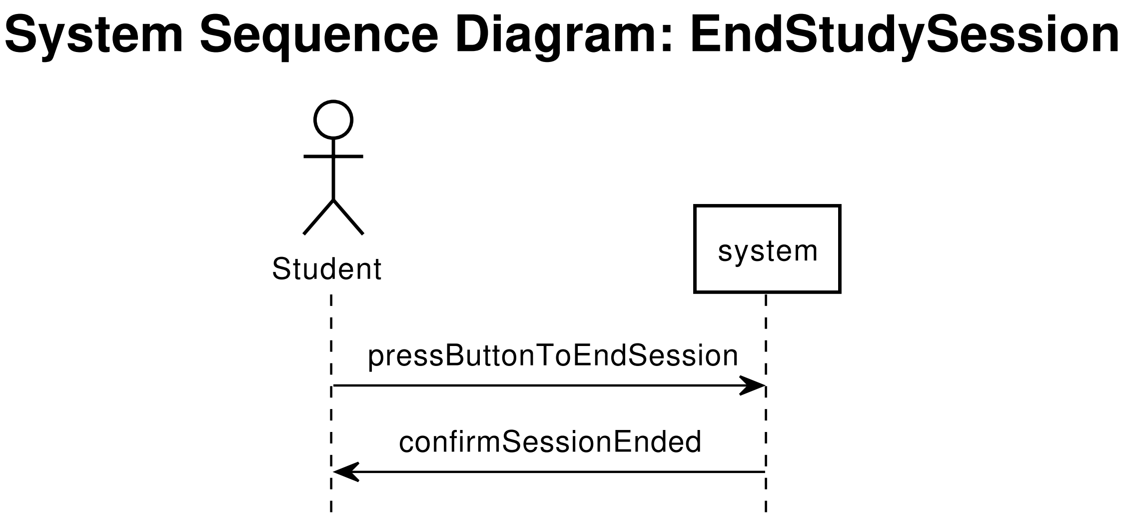



**Figure 3:** SSD: End Study Session

Figure 2.3: System Sequence Diagram for IoT-initiated session termination.

### Architectural Utility and Firmware Control Loops

These two SSDs guided the design of the device's main loop (`main.c`) and API integration layer (`server_api.c`) in several ways: 1. **Watchdog Self-Healing Routine**: The start sequence highlights a synchronous POST request to `/session`. If the network fails or the server returns an error, the firmware triggers a hardware Watchdog reset (`wdt_enable(WDTO_30MS)`) after 5 retries, forcing a clean device reboot. This self-healing mechanism is directly derived from modeling the system boundary response to connection failures. 2. **Button Cooldown and Debouncing**: To prevent double-triggering or race conditions on the server (creating multiple ghost sessions for a single physical press), we implemented a software-based timer cooldown (`session_button_cooldown_timer`) of 10 seconds. This state locking is reflected in the SSD transitions where session commands are serialized.

## 2.4.2 Active Telemetry and Real-Time Alerting

During a running session, the system must ingest telemetry and evaluate the environment without blocking the microcontroller or introducing dashboard delays.

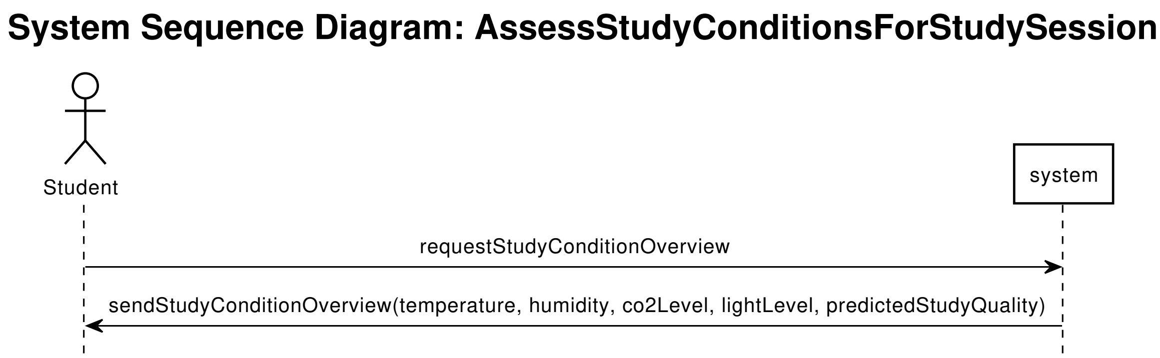



**Figure 4:** SSD: Assess Study Conditions

Figure 2.4: System Sequence Diagram for active telemetry ingestion.

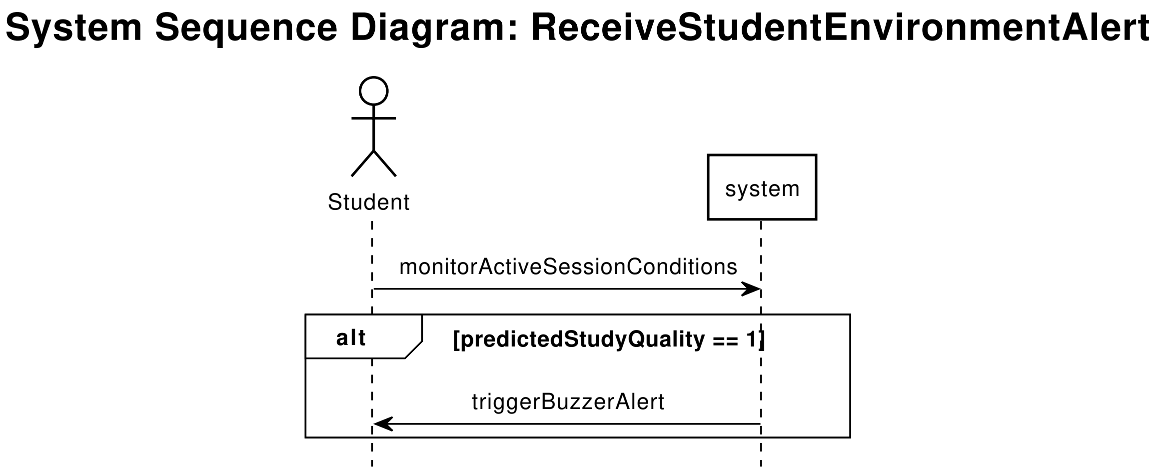



**Figure 5:** SSD: Receive Environment Alert

Figure 2.5: System Sequence Diagram for real-time environment alerts.

### Ingestion and Alerting Execution Details

1. **Asynchronous Dual-Timer Routine**: The firmware utilizes two distinct software-based interrupts. A 5-second timer (`pulse_timer`) posts keep-alives to `/session/{id}/pulse` to maintain session persistence on the server. A 30-second timer (`data_timer`) reads physical sensors (DHT11, light ADC, MH-Z19 CO2 UART) and transmits telemetry to `/data`. Modeling these as separate SSD streams allowed us to decouple connection validation (keep-alives) from resource-heavy sensor polling.
2. **Direct Feedback Loop**: The response from `/data` returns the ML-predicted study suitability score. The SSD shows that if the returned score is less than 4 (poor or moderate comfort), the device enters a local buzzer alerting loop (`buzzer_beep()`). Modeling this response loop as a direct system output payload prevented the need for the device to perform local calculation or pull data, saving processing power and memory on the ATmega2560.

## 2.4.3 Comfort Feedback and Prediction Loop

After a study session, the student evaluates their comfort experience.

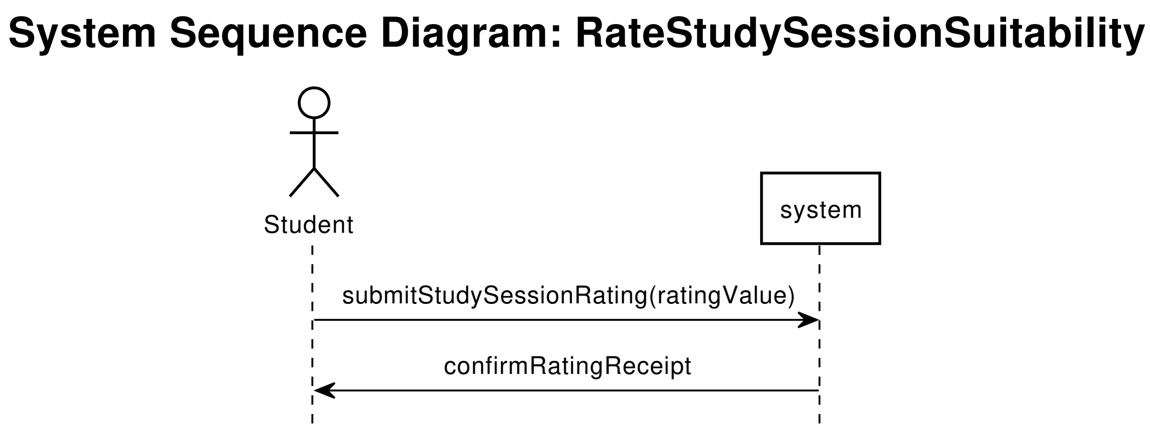



**Figure 6:** SSD: Rate Study Session Suitability

Figure 2.6: System Sequence Diagram for post-session comfort feedback.

### Feedback Loop and API Specification

The rating sequence maps the transition of user feedback into training labels: 1. **API Validation Contract**: The SSD defines a single transaction `submitStudySessionRating(ratingValue)` to `/ratings`. This guided the definition of the FastAPI Pydantic schema (`RatingRequest`), enforcing that rating values remain between 1 and 5 in the client payload. 2. **Frontend UI State Mapping**: By modeling the receipt confirmation, the frontend React application (`SessionRating.jsx`) is structured to reset local states (clearing the rating selection) and close the modal container only upon receiving a successful status code from the backend.

## 2.5 Use Case Control Flow and State Transitions

To ensure consistency in state handling across the IoT device firmware, web frontend, and cloud backend, we mapped the system's operational control flow.

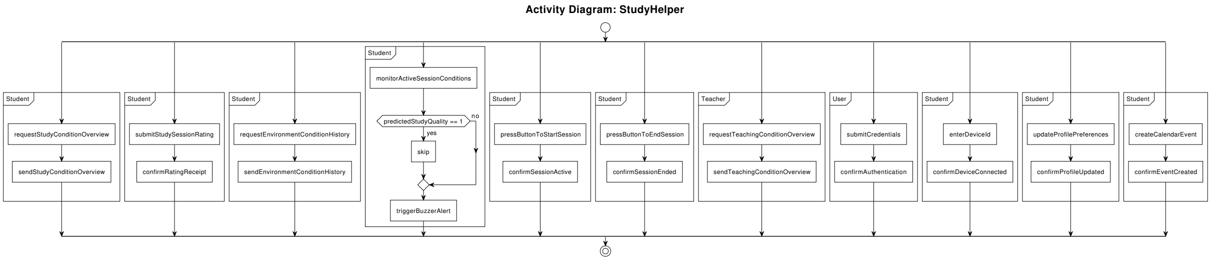



**Figure 7:** Use Case Control Flow

Figure 2.7: Activity Control Flow Diagram of the StudyHelper Use Cases.

## System-Wide State Synchronization and Validation

Modeling the parallel execution of these transactions helped the development team resolve synchronization bottlenecks across different execution environments:

1. **Firmware State Transition and Lockouts**: When a student triggers `StartStudySession` or `EndStudySession` via the physical button, the device transitions its internal state. The activity flow visualizes how this triggers a backend transaction, updating the PostgreSQL database. The web application synchronizes with this state via SSE/polling, displaying active monitoring statuses. By tracking the control flow, we realized that starting a session should lock out certain options (like taking one-time measurements via button 2), protecting the UART sensor serial bus from overlapping requests.
2. **Dynamic Alerting Flow**: The alert flow (modeled under `ReceiveStudentEnvironmentAlert`) shows that during a session, the system continuously reads sensor telemetry. The alert sequence validates the ML-predicted comfort index and decides whether to skip or trigger the buzzer alarm. Mapping this flow clarified that alert validation should happen synchronously on the backend during telemetry ingestion, avoiding client-side alert calculation.
3. **Session Ratings during Active Sessions**: Originally, comfort ratings were conceptualized as occurring only after a session had terminated. By mapping the parallel execution branches in the activity diagram, we realized comfort ratings could be submitted during an active session as comfort levels fluctuate. This motivated us to allow comfort ratings during a session, which allows multiple rating inputs to be associated with different telemetry intervals.

# 3. DESIGN

## 3.1 Design Overview

Authors: [Name, Name, Name]

### 3.1.1 System Architecture

[Describe the overall architecture: IoT device → Cloud backend → Frontend, with the ML pipeline integrated into or alongside the cloud layer. Reference the architecture diagram:]

[Justify the key architectural decisions: why this decomposition, how components communicate (protocols, data formats), and how data flows end-to-end.]

The StudyHelper system follows a layered, distributed architecture comprising four runtime components: the embedded IoT firmware, the Core API, the MAL API, and the React frontend. All server-side components are deployed as Docker containers behind a Coolify-managed reverse proxy (Caddy/Traefik) and share a single PostgreSQL database, as described by `docker-compose.yml`.

The IoT device sits at the edge of the system. It collects sensor readings and transmits them over HTTP to the Core API, acting as the sole source of real-world environmental data. This unidirectional push pattern was chosen because the device does not need to receive commands in normal operation and because HTTP over TCP is straightforward to implement with the ESP8266-based Wi-Fi module used alongside the ATmega2560.

The Core API (FastAPI) is the system's central gateway. It handles device registration, session lifecycle management, sensor data persistence, and user authentication. It is the only component with direct write access to the PostgreSQL database. The Core API also acts as a proxy to the MAL API for suitability predictions - forwarding prediction requests from the frontend and returning the result - so that the frontend has a single API surface to communicate with. Separating the prediction concern into the MAL service means the ML component can be retrained and redeployed independently without affecting the Core API.

The MAL API (FastAPI) is responsible for model inference and data collection for retraining. It receives prediction requests forwarded by the Core API, loads the trained model artifact, and returns a suitability rating. It also exposes endpoints for exporting raw sensor data from the database to support offline retraining workflows.

The React frontend is a static single-page application served by Nginx inside a Docker container. It communicates exclusively with the Core API via REST calls - for sensor history, user authentication, and suitability predictions. All inter-service communication uses JSON over HTTP. The Coolify reverse proxy handles TLS termination and routes public traffic to the frontend and Core API containers.

Data flows through the system as follows: the IoT device pushes a sensor payload to the Core API every thirty seconds; the Core API persists the reading in PostgreSQL; the frontend polls the Core API for the latest readings and requests a prediction, which the Core API forwards to the MAL API; the MAL API returns a rating; the Core API relays it to the frontend, which renders the result.

[TODO: Insert the architecture diagram reference here once the SVG is exported.]

### 3.1.2 Cloud Architecture

The StudyHelper backend is hosted on **Coolify**, an open-source self-hosted PaaS running on a public VPS. Coolify provides container lifecycle management and a built-in reverse proxy (Caddy/Traefik) that terminates TLS and routes public traffic to the frontend and Core API services. Internal traffic between services stays on the Docker network.

**Containerisation:** All runtime services are defined in `docker-compose.yml` and run as Docker containers: `postgres`, `api`, `mal-api`, and `frontend`. The `api` service is built from `API/Dockerfile` (Python 3.12) and serves the Core API with Uvicorn on port 8080. The `mal-api` service is built from `MAL/Dockerfile` and serves the ML inference API on port 8000, including the committed Random Forest[TODO: update after final model] model artifact (`models/rf_model.pkl`). The `frontend` service is built from `Frontend/Dockerfile`, compiles the Vite/React application, and serves static assets with Nginx on port 80. The `postgres` container uses the official `postgres:16-alpine` image with a named volume (`postgres_data`) for persistence.

**Database:** A single PostgreSQL 16 instance is the shared persistence layer. The schema is initialized via `initdb/01_schema.sql` mounted as an entrypoint init script. The Core API owns write access for devices, sessions, and sensor data, while the MAL API reads data for export and model support.

**RESTful API design:** The IoT device communicates directly with the Core API over HTTP (no message queue is used). Device registration uses `POST /device`, sessions use `POST /session` and `PATCH /session/{id}/pulse`, and sensor submissions use `POST /data`. The frontend communicates exclusively with the Core API for readings and predictions; prediction requests are forwarded by the Core API to the MAL API `POST /predict` endpoint. All inter-service payloads are JSON.

**Serverless workloads:** No separate serverless runtime is deployed. Data extraction and model verification tasks are handled by on-demand GitHub Actions workflows (for example, `data-extract.yml`).

**Continuous delivery:** On every push to `main`, a GitHub Actions workflow sends a webhook to Coolify, which pulls updated images from GHCR and restarts the affected containers. Pre-built images for `mal-api` and `frontend` are published to GHCR as part of the MLOps and frontend CI/CD pipelines.

### 3.1.3 Security Design

**IoT–cloud encryption:** [Describe the encryption mechanism used between the ATmega2560 firmware and the IoT backend. Specify whether TLS is terminated at the Coolify reverse proxy, whether a certificate is used, and how the device authenticates itself to the server - e.g. via a pre-shared device ID string in the JSON payload or via a bearer token in the HTTP header.]

**API authentication:** The MAL API export endpoint (`GET /export-data`) is protected by an `X-Export-Token` HTTP header checked against the `MAL_API_EXPORT_TOKEN` environment variable. This prevents unauthorised access to raw sensor exports. [Describe the authentication mechanism for the IoT backend endpoints — the `secrets.ini.example` file in the IoT firmware references a `SECRET_KEY` environment variable on the API container, which suggests token-based authentication is configured. Confirm whether JWT is implemented and which endpoints require it.]

**Secret management:** Sensitive credentials - database password, Wi-Fi SSID and password, server hostname, device ID, and the API secret key - are stored as environment variables, never hard-coded in source files. The firmware secrets are injected at build time via `secrets.ini`, which is excluded from the repository by `.gitignore`. Server-side secrets are passed to Docker containers via the `.env` file, which is similarly excluded. The `docker-compose.yml` uses mandatory variable substitution (`:?` syntax) so that a deployment will fail explicitly if any required secret is missing rather than silently using an empty value.

**Additional considerations:** Because the ATmega2560 has limited cryptographic hardware support, the choice of encryption scheme involves a trade-off between security strength and memory and processing overhead. [Justify the chosen approach in light of these constraints.] The database is not exposed on a public port; access is restricted to the Docker internal network, which limits the attack surface from the public internet.

[Complete this section with any additional security measures implemented, such as rate limiting on the prediction endpoint, CORS configuration on the backends, or HTTPS enforcement by the Coolify reverse proxy.]

### 3.1.4 DevOps Strategy

Reference the repository: [GitHub/GitLab URL]]

The StudyHelper project follows a trunk-based branching model with two long-lived protected branches: `main` (production) and `dev` (integration). Feature and bugfix work is developed on short-lived branches cut from `dev` and merged via pull requests. Direct pushes to `main` are blocked; all changes reach `main` through a PR that has passed the required CI checks for its component area. Pair-programming sessions are documented in commit messages by listing both contributors in the format `Co-authored-by: Name <email>`, consistent with GitHub's co-authorship convention.

**CI/CD overview:** Three separate GitHub Actions workflows cover the three main codebases:

- **IoT CI** (`iot-ci.yaml`): triggered on pull requests to `main` and `dev` that modify files under `IOT/`. Runs two sequential jobs: native unit tests compiled with GCC and coverage reporting via lcov, followed by firmware compilation for the ATmega2560 using PlatformIO. A flashable `firmware.hex` artifact is published on every passing build.
- **MLOps** (`mlops.yaml`): triggered on pull requests to `main` and `dev`. Runs the full `pytest` suite for data pipeline and API tests, verifies that `rf_model.pkl` [TODO:update]is present and loadable, and on merge to `main` builds and pushes the `mal-api` Docker image to GHCR.
- **Frontend CI/CD** (`frontend.yaml`): [Describe the frontend pipeline — what checks run (lint, tests, build) and whether the frontend image is pushed to GHCR on merge.]
- **Deployment** (`deploy-coolify.yaml`): triggered on pushes to `main`. Sends a webhook to Coolify to redeploy the production stack.

**Container strategy:** All services are containerised. The IoT backend (`api`) uses a multi-stage .NET Dockerfile. The MAL API (`mal-api`) uses a Python Dockerfile that installs dependencies from `requirements.txt` and copies the model artifact. The frontend (`frontend`) uses a Node.js build stage and an Nginx serve stage. The database uses the unmodified `postgres:16-alpine` official image.

**Testing overview:** Unit testing is component-specific. The IoT firmware tests cover the `wifi_http` and `server_api` modules using Unity and FFF, running natively on the CI host. The ML pipeline tests cover data transformation logic and the FastAPI prediction endpoint using pytest. [TODO: Describe the frontend test approach — e.g. whether Jest or React Testing Library tests are included in the pipeline.] Integration testing across components is manual, performed by flashing the firmware, running the backend stack locally, and observing end-to-end data flow.

The repository is hosted at [insert GitHub URL]. [Add any tag naming conventions used for releases, e.g. `v1.0.0` semantic versioning tags on `main`.]

## 3.2 IoT Design

Authors: [Name, Name]

## 3.2 IoT Design

Authors: [Jakub Maciej Baczek, Tymoteusz Krzysztof Zydkiewicz]

The IoT component is the physical sensing layer of the StudyHelper system. Its main responsibility is to collect indoor environmental data from the study space and transmit it to the backend, where the readings can be stored, displayed in the frontend, and used as input for machine-learning-based study suitability prediction.

The design is based on three central requirements. First, the device must measure the environmental factors selected for the project: temperature, humidity, $CO_2$ concentration, and light level. Second, it must support a simple study-session workflow controlled through physical buttons, so the user can start and stop monitoring without interacting with the web interface. Third, it must communicate with the backend using structured HTTP requests over Wi-Fi.

The IoT design therefore combines sensors, user input, local status feedback, and network communication in one embedded prototype. The firmware is divided into small modules with clear responsibilities, so that hardware drivers, backend communication, display status logic, and the main session controller can be developed and tested separately.

### 3.2.1 Hardware Architecture

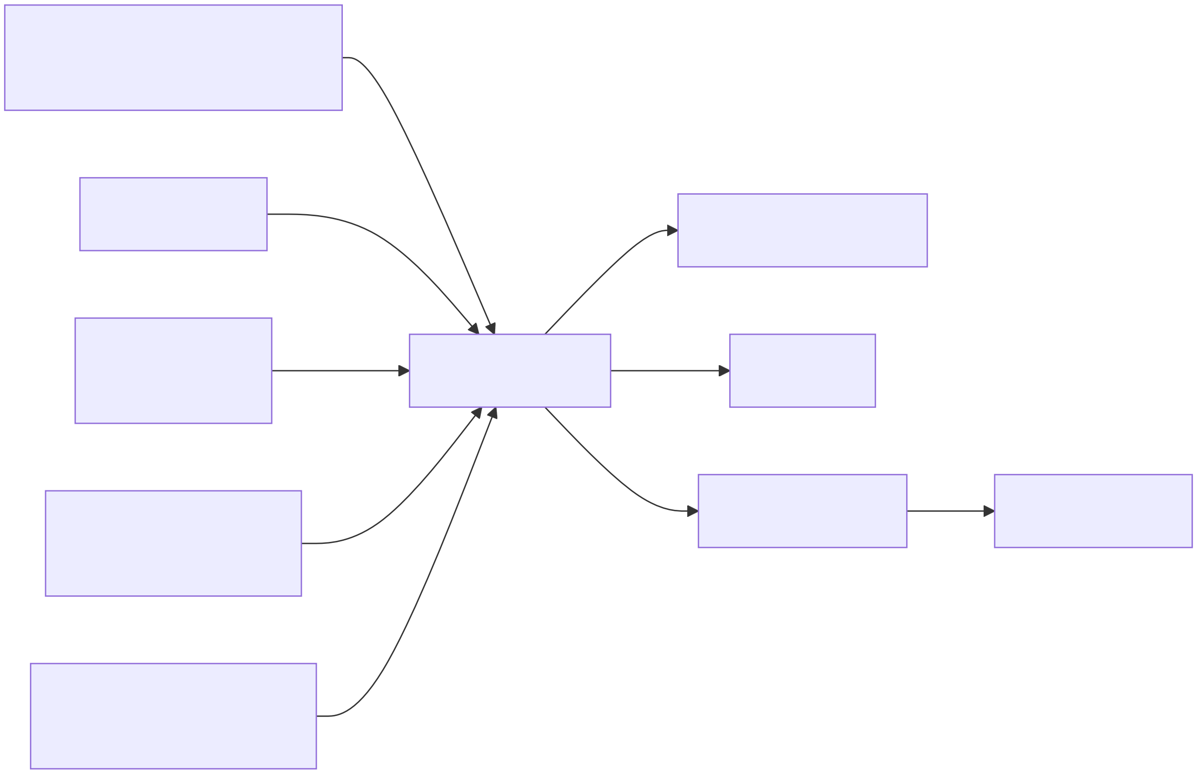


Figure 3.x: Hardware block diagram of the IoT device.

The hardware architecture is centred around an ATmega2560-based microcontroller board. The ATmega2560 was chosen because it provides sufficient GPIO pins, ADC channels, timers, and UART interfaces for a multi-sensor embedded prototype, while remaining compatible with the Arduino and PlatformIO development environment used in the project.

The device uses the following main hardware components:

| Component | Interface | Purpose |
| --- | --- | --- |
| ATmega2560 microcontroller | GPIO, ADC, UART, timers | Main controller for sensors, buttons, display, buzzer, and communication |
| DHT11 temperature and humidity sensor | Single-wire digital GPIO | Measures air temperature and relative humidity |

| Component | Interface | Purpose |
|---|---|---|
| MH-Z19B CO2 sensor | UART3 at 9600 baud | Measures CO2 concentration in ppm |
| KY-018 light sensor | ADC channel PK7 | Measures ambient light level |
| Wi-Fi module | UART / AT commands | Provides TCP/IP communication with the backend API |
| Button 1 | GPIO input with pull-up | Starts and stops a study session |
| Button 2 | GPIO input with pull-up | Triggers an instant measurement mode |
| Four-digit display | Shift-register style GPIO output, refreshed by timer interrupt | Shows local device state |
| Buzzer | GPIO output | Alerts the user when predicted study quality is poor |

The selected sensors match the environmental factors used by the StudyHelper system. The DHT11 provides temperature and humidity, the MH-Z19B provides CO2 concentration, and the KY-018 light sensor provides a simple analogue measure of room brightness. The light driver inverts the raw ADC value so that higher values represent brighter conditions, making the reading easier to interpret in the rest of the system.

The CO2 sensor requires a different interaction pattern from the DHT11 and light sensor. The firmware sends a read command to the sensor and receives a nine-byte UART response frame. The frame is validated with a checksum before the measured ppm value is accepted. This design avoids using invalid or corrupted CO2 readings in the transmitted sensor payload.

The device also includes local user interaction and feedback. Button 1 controls the study-session lifecycle. Button 2 is used for instant measurement mode when no session is active. The four-digit display shows status patterns for boot, idle, active-session, and instant-measurement states. The buzzer is used as an alert mechanism when the backend returns the lowest study quality rating.

Sound measurement was considered during the project, but it is not part of the final active IoT design because the available sound sensor did not provide reliable enough readings. The final hardware design therefore focuses on temperature, humidity, CO2, and light, which are the sensor values actually transmitted by the firmware.

### 3.2.2 Embedded Software Architecture

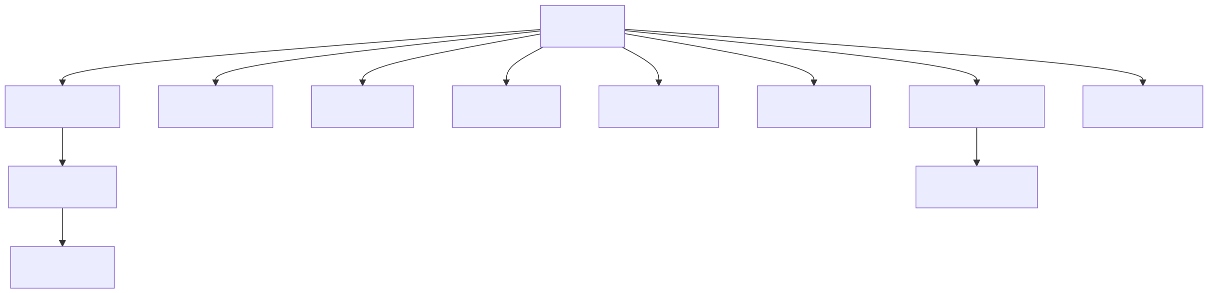


Figure 3.x: Firmware module structure for the IoT component.

The embedded software is implemented in C for the ATmega2560 and follows a modular architecture. The `main.c` file coordinates the application flow, while individual modules handle backend communication, HTTP request construction, display status patterns, sensors, buttons, timers, and local alerts.

The most important firmware modules are:

| Module | Responsibility |
|---|---|
| `main.c` | Coordinates boot, Wi-Fi setup, button handling, timers, session state, and sensor reading |
| `server_api.c` / `server_api.h` | Implements backend-specific actions such as device registration, session creation, keepalive pulses, data upload, instant prediction requests, and local alert handling |
| `wifi_http.c` / `wifi_http.h` | Resolves the backend host, creates TCP connections, builds HTTP requests, transmits payloads, and reads responses |
| `dht11.c` | Reads temperature and humidity from the DHT11 sensor |
| `co2.c` | Handles UART communication and checksum validation for the CO2 sensor |
| `light.c` | Wraps ADC access for the light sensor |
| `button.c` | Reads physical button states with pull-up inputs |
| `timer.c` | Provides software timers used for periodic session actions |

| Module | Responsibility |
| --- | --- |
| `display_status.c` | Maps firmware states to four-digit display patterns |
| `buzzer.c` | Produces local audio alerts |

The firmware uses a cooperative superloop design. After startup, the program continuously checks button states and timer flags. Time-critical and periodic behaviour is represented by flags such as `pulse_due` and `data_due`, which are set by timer callbacks and then handled in the main loop. This keeps the main control flow explicit and avoids introducing a full real-time operating system, which would be unnecessary for the scope of the device.

Three software timers define the main runtime schedule:

| Timer | Interval | Purpose |
| --- | --- | --- |
| Pulse timer | 5 seconds | Sends a keepalive pulse while a session is active |
| Data timer | 30 seconds | Sends environmental measurements while a session is active |
| Button cooldown timer | 10 seconds | Prevents accidental repeated start/stop actions |

The device communicates with the backend using HTTP over TCP through the Wi-Fi module. HTTP was chosen because the rest of the StudyHelper system exposes REST-style endpoints and JSON payloads. This keeps the interface between the IoT firmware and backend simple, transparent, and easy to test. Each request opens a TCP connection, sends an HTTP request, waits for a response for up to three seconds, and then closes the connection.

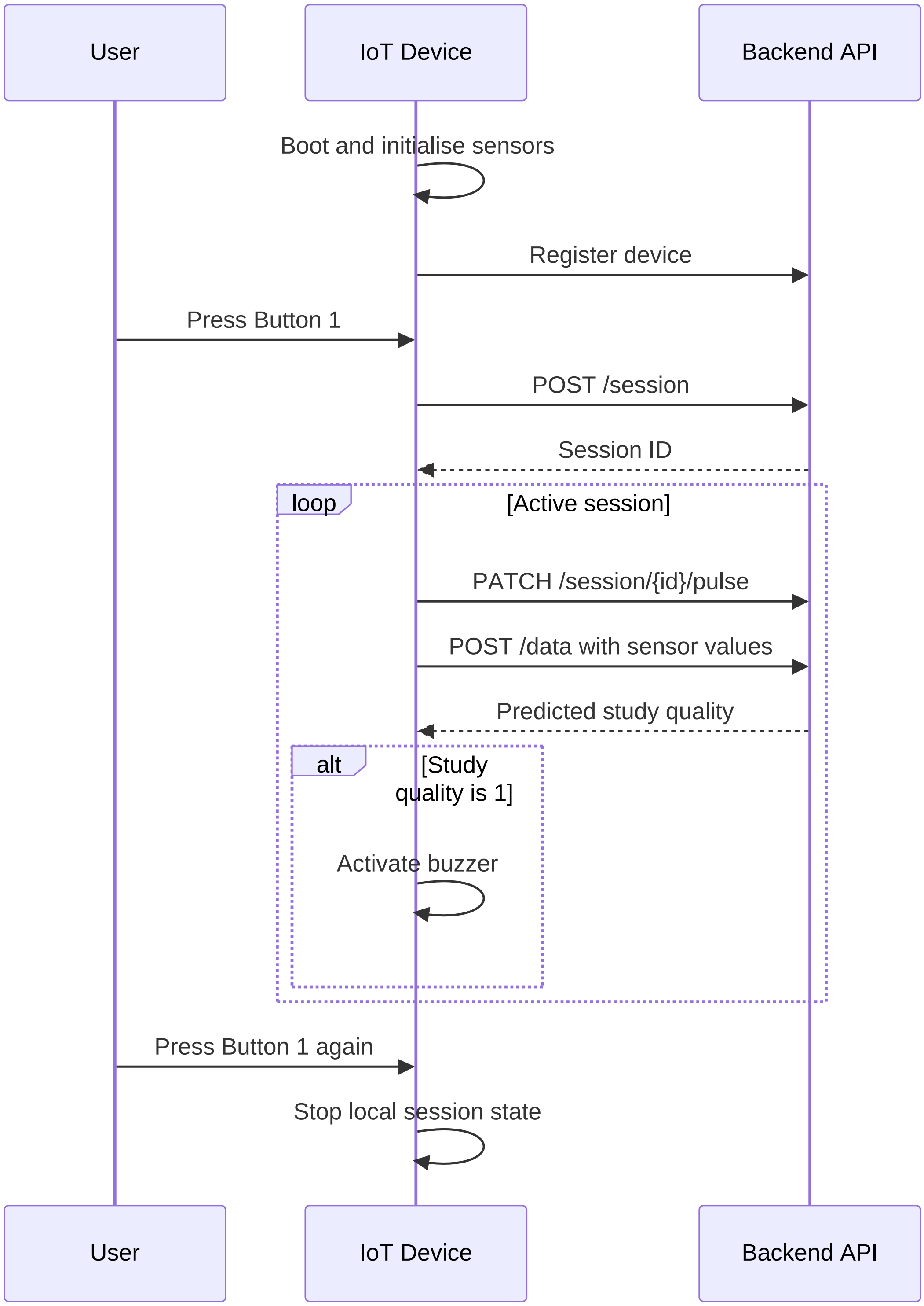



Figure 3.x: Session flow between the user, IoT device, and backend API.

The session flow begins during boot. The firmware initialises the display, UART, sensors, Wi-Fi module, and backend host resolution. It then registers the physical device ID with the backend. After this setup, the device enters idle mode and waits for user input.

When Button 1 is pressed, the firmware sends a `POST /session` request containing the configured device ID. If the backend returns a valid session ID, this ID is stored locally and used for later requests. During an active session, the device sends `PATCH /session/{id}/pulse` every five seconds and `POST /data` every thirty seconds. The data payload contains the session ID, temperature, humidity, light level, and CO2 level in JSON format.

The firmware stores request and response data in statically allocated buffers. This design avoids dynamic memory allocation and reduces the risk of heap fragmentation on the ATmega2560. Watchdog resets are performed during network waiting periods so that normal communication delays do not incorrectly reset the device, while still protecting the firmware from longer hangs.

If the backend reports that a session is no longer alive, the firmware clears the local session ID and attempts to start a new session. If a sensor data response contains a study quality rating of `1`, the buzzer is activated to warn the user that the current conditions are poor. This creates a local feedback loop where the IoT device does not only collect data, but also reacts to the study suitability prediction returned by the system.

The firmware also includes an instant measurement mode controlled by Button 2. This mode allows the device to take a one-time measurement and request a prediction without entering the normal periodic session loop. It was designed as a quick check of the current environment and reuses the same sensor-reading and backend-communication modules as the session workflow.

## 3.3 Machine Learning — Data Exploration

Authors: [Name, Name]

### 3.3.1 Data Sources and Collection Strategy

The initial search for data focused on the relationship between environmental noise and cognitive focus. We explored combining datasets describing focus-related effects of background noise with labeled sound categories from WAV files, extracting frequencies and loudness. However, as the project evolved, we narrowed the ML objective from direct focus prediction to predicting a user-provided **Study Suitability Rating**. This shift was necessary because “focus” is a subjective internal state that cannot be directly measured by our sensors. By using a user-provided rating, we anchored our target variable in observable environmental conditions and explicit user feedback. […]

### 3.3.2 Exploratory Data Analysis

During the exploratory phase, several candidate datasets were evaluated. We identified and eliminated datasets that appeared synthetic or "too perfect" to be realistic sensor data. For example, in datasets labeled as "Data 2" and "Data 4," the distributions of humidity, noise, and light were suspiciously uniform or perfectly bell-shaped, lacking the stochastic noise typical of real-world environments.

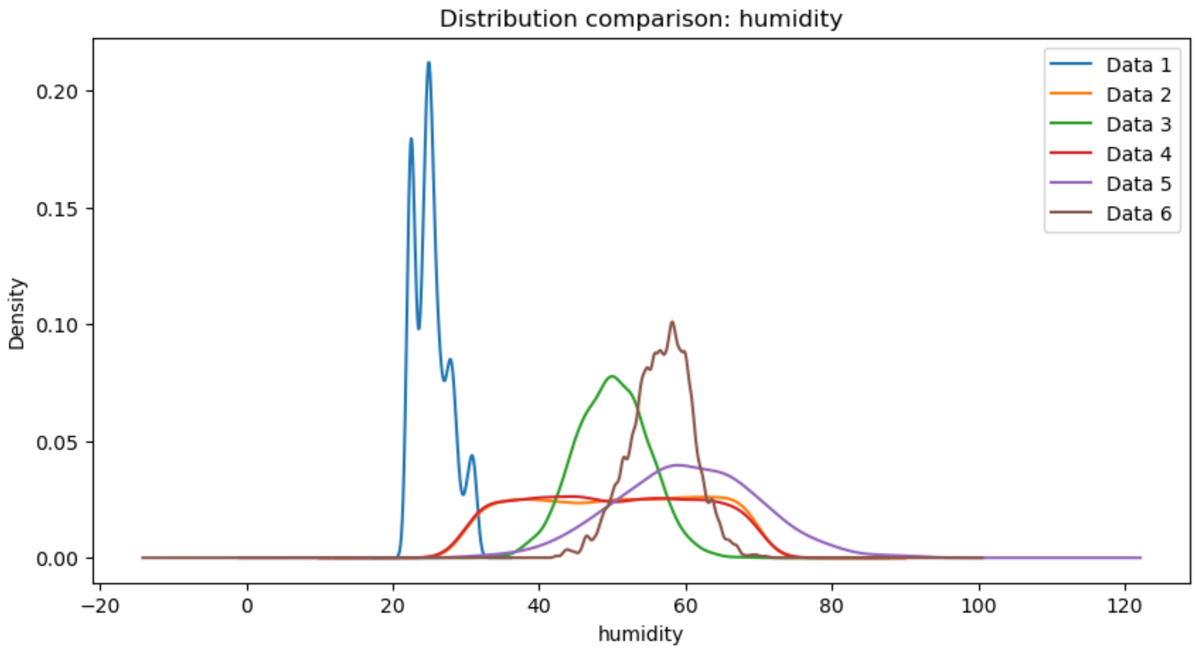



**Figure 8:** Feature Distributions of Suspicious Datasets

Figure 3.x: Suspiciously perfect feature distributions in candidate datasets.

Correlation analysis further revealed insights into data quality. Healthy datasets exhibited natural correlations between temperature, $CO_2$, and humidity. Conversely, some "suspicious" datasets showed near-zero correlation across all features, suggesting high randomness or artificial generation.

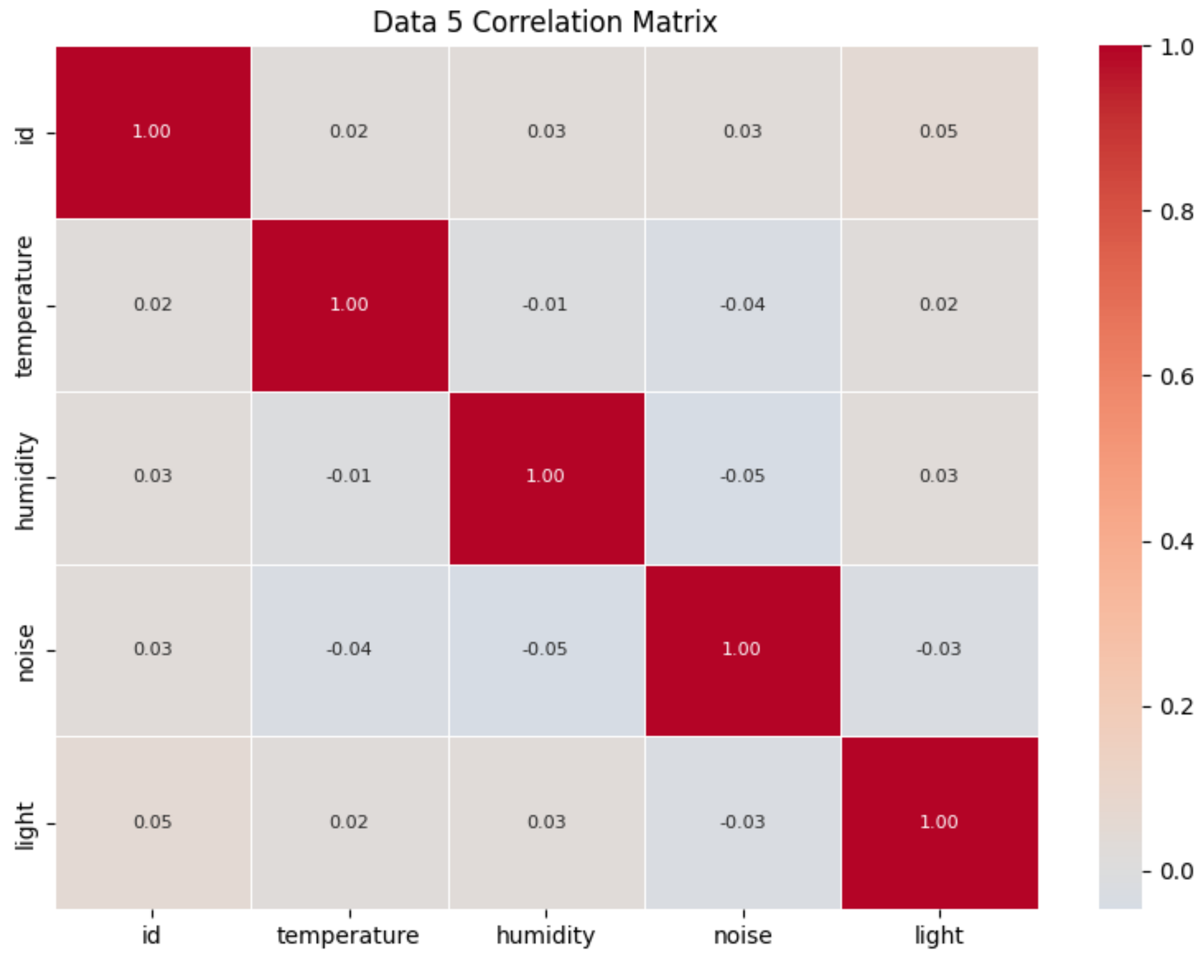



**Figure 9:** Healthy vs Suspicious Correlation Matrices

Figure 3.y: Correlation matrices showing healthy physical relationships (left) vs suspicious randomness (right).

## 3.3.3 ML Problem Formulation

The ML task is formulated as a supervised learning problem aimed at predicting the Study Suitability Rating. Specifically, this is defined as a **Multi-Class Classification** problem:

- **Inputs (X):** The features are derived from continuous, time-series sensor telemetry (Temperature, Humidity, CO2, and Light). These raw readings are dynamically aggregated into structured vectors (capturing current, minimum, maximum, and mean values) to summarize the state and variance of the environment over an active session.

- **Output (Y):** The target variable is the Study Suitability Rating, consisting of discrete integer classes ranging from 1 (poor suitability) to 5 (excellent suitability).
- **Objective:** The goal is to learn the complex, non-linear relationships between the physical characteristics of a room and subjective human sustainability raiting. By mapping environmental metrics to historical user ratings, the model enables the system to automatically predict the quality of an environment in real-time.

## 3.4 Frontend Design

Authors: [Name, Name]

### 3.4.1 UI/UX Design

[Describe the user interface design:

- Target users and their primary tasks
- Navigation structure and information architecture
- Key screens/views and their purpose]

### 3.4.2 Component Architecture

[Describe the React component hierarchy. How is the application decomposed? What state management approach is used (useState, Context API, Redux, Zustand)? How does the app communicate with the backend API (fetch, axios, React Query)?]

### 3.4.3 Responsiveness Strategy

[Describe the responsive design approach:

- CSS framework or approach (Tailwind, Bootstrap, CSS Grid/Flexbox)
- Breakpoints used and how layouts reflow at each
- Specifically address the three mandatory widths: 576px, 768px, 1200px]

## 3.5 IoT Implementation

Authors: [Damian Michal Choina]

### 3.5.1 Sensor and Actuator Drivers

The firmware uses a layered driver structure: each peripheral is encapsulated behind a small header in `IOT/lib/` that exposes a minimal initialisation function and one or two read or write operations. This separation keeps the application code in `main.c`, `server_api.c`, and `wifi_http.c` free of register-level concerns and makes individual drivers replaceable without touching the application logic.

Four sensors and three actuator-class peripherals are integrated. The **DHT11** temperature and humidity sensor exposes a blocking call `dht11_get()` that returns four values — humidity integer and decimal parts, temperature integer and decimal parts — along with a status code, so the application can skip transmission on a failed read. The **MH-Z19B** $CO_2$ sensor communicates over UART and is driven by two separate calls: `co2_request_measurement()` triggers a new reading, and `co2_read_ppm()` retrieves the latest verified value once available. Because the sensor needs time to respond between request and reply, the two calls are scheduled on alternating data cycles in the main loop - if a fresh reading is not yet available, the last cached value is reused so the transmission cadence is never blocked by sensor latency. The **KY-018** light sensor is read via `light_measure_raw()`, which returns a 10-bit value already scaled so that low means dark and high means bright. Moreover, no further calibration is applied in firmware, since interpretation is left to the server.

The remaining peripherals are simpler. **Two pushbuttons** are polled via `button_get()` on every iteration of the main loop and debounced via a 50 ms re-read pattern rather than dedicated interrupts. A **four-digit 7-segment display** is driven by the low-level `display_setValues()` and `display_setDecimals()` functions; on top of these, the `display_status` module — written as part of this project — exposes four named patterns (`boot`, `idle`, `session`, `instant`) so the application does not have to know about individual digit codes. A **buzzer** is exposed as a single `buzzer_beep()` call producing a short tone; longer alerts are produced by repeating the call, used to signal unfavourable study conditions — the buzzer sounds when the server returns a `study_quality` value below 4. Finally, a **software-timer abstraction** in `timer.h` provides callback-based timers, allowing the application to register callbacks for pulse, data, and button-cooldown events without managing hardware timers directly.

The $CO_2$ scheduling pattern in the main loop illustrates how the application accommodates sensor-side latency without blocking. On every data cycle, the most recent reading is consumed (if one has become available), the cached value is updated, and a new measurement is immediately requested for the next cycle:

```
if (co2_read_ppm(&current_co2) == CO2_OK) {
    latest_co2_ppm = current_co2;    /* fresh reading — update cache */
} else {
```

```
    /* no new reading yet — reuse last cached value */
}
co2_request_measurement();           /* request next reading */
```

### 3.5.2 Cloud Communication Implementation

Network communication is split into two layers. The lower layer (`wifi_http.c`) is responsible for the HTTP transport — DNS resolution, TCP connection lifecycle, and request formatting. The upper layer (`server_api.c`) is responsible for protocol concerns — endpoint paths, JSON payloads, response parsing, and session state. This separation keeps the HTTP layer reusable and the protocol layer free of low-level transport details.

At boot, `http_resolve_host()` calls `wifi_command_get_ip_from_URL()` once to translate `SERVER_HOST` into an IPv4 address, which is then cached in a static buffer so each subsequent request avoids a DNS round-trip. The main loop retries DNS until it succeeds, treating it as a hard prerequisite for normal operation. Each HTTP request is built into a single 384-byte stack buffer using `snprintf` and handed to the Wi-Fi driver via `wifi_command_TCP_transmit()`. A fixed-format request is used (HTTP/1.0, `Connection: close`, JSON body):

```
snprintf(req, sizeof(req),
         "%s %s HTTP/1.0\r\n"
         "Host: %s:%d\r\n"
         "Content-Type: application/json\r\n"
         "Content-Length: %u\r\n"
         "Connection: close\r\n"
         "\r\n%s",
         method, endpoint, SERVER_HOST, SERVER_PORT,
         (uint16_t)strlen(body), body);
```

Failures are handled defensively at every step. A failed TCP connect or transmit closes the connection, waits 500 ms with watchdog resets, and returns the error to the caller. After a successful send, the loop busy-waits up to 3 000 ms for the server's response — again with watchdog resets — and proceeds even if no response arrives, so the device never deadlocks on a silent server. All buffers are statically allocated; nothing on the network path uses the heap.

At the protocol layer, `server_api.c` implements three workflows: one-time device registration (`POST /device`), session lifecycle (`POST /session`, `PATCH /session/{id}/pulse`), and data reporting (`POST /data`, `POST /instant-measurement`). Session start is wrapped in a retry loop of up to five attempts with a 2-second back-off and if the server cannot be reached or its response cannot be parsed within that budget, the watchdog is deliberately enabled with a short timeout to force a clean reboot. This recovery pattern — retry, then reboot — was chosen because a partially-initialised device with no session ID has no meaningful path forward. Pulse responses are also inspected: if the server replies with `"alive":false`, the device assumes the session was lost and transparently restarts it without user intervention.

### 3.5.3 Main Application Logic

The main application is a single cooperative loop in `main.c`. After hardware and Wi-Fi initialisation are complete and the device has registered with the backend, the loop runs forever and performs four responsibilities on every iteration: poll the buttons, dispatch any button action, check time-driven flags, and dispatch the corresponding periodic server transmission.

Two pieces of time-driven behaviour are required during an active session: a pulse every 5 seconds to keep the server-side session alive, and a data submission every 30 seconds carrying current sensor readings. Rather than performing these transmissions directly from interrupt handlers — where blocking HTTP calls would be unsafe — the software timer system raises two volatile flags, `pulse_due` and `data_due`, which are consumed at a safe point in the main loop:

```
if (session_active && !request_in_progress && data_due) {
    data_due = 0;
    request_in_progress = 1;
    /* read sensors, then server_send_data(...) */
    request_in_progress = 0;
}
```

The `request_in_progress` flag acts as a simple mutex. While one HTTP request is in flight, no other request can be started and no button press can interrupt it. This avoids re-entering the Wi-Fi driver with overlapping commands, which the underlying ESP8266-based module does not tolerate.

User input is handled in the same loop. Button 1 toggles between the idle and active session states; Button 2 triggers an on-demand instant measurement and is only operative outside an active session. Both buttons are debounced via a two-read pattern with a 50 ms delay between samples. An additional 10-second cooldown timer prevents the user from rapidly toggling the session on and off, both because the underlying HTTP work is expensive and to give the server's session lifecycle time to settle. State transitions are mirrored on the 7-segment display through the `display_status` module — `boot`, `idle`, `session`, and `instant` — so that the visible state of the device is always consistent with its internal state without scattering display logic across the file.

# 3.6 Machine Learning — Preprocessing and Pipeline

Authors: [Name, Name]

## 3.6.1 Data Cleaning and Imputation

A significant challenge was merging disparate datasets, which often resulted in missing columns for specific sensors (e.g., noise or light). To handle these missing values while preserving natural variance, we implemented a sophisticated imputation strategy using the **Multivariate Imputation by Chained Equations (MICE)** framework.

Rather than using simple means or linear regression, we adopted a cluster-based approach:

1. **Environment Type Clustering**: We used k-means clustering to group data points into "environment types" based on features that were fully present (Temperature, CO2, Humidity). This accounts for different physical environments (e.g., sun-exposed sessions vs. windowless labs sessions) where sensor correlations might differ.
2. **ExtraTrees Estimator**: Within the MICE framework, we utilized an ExtraTrees estimator to model non-linear relationships.
3. **Variance Preservation**: We modified the imputation logic to include natural distribution variance based on the average standard deviation from the trees, preventing the "flat average" effect.

If a cluster suffered from extreme sparsity (e.g., completely missing a feature like noise), a global median was used as a fallback to prevent model bias.

## 3.6.2 Feature Selection

Our feature extraction pipeline aggregates ("linearize") time-series sensor data into session-level metrics that are calculated dynamically for the active session. This generates a comprehensive 16-feature vector encompassing current (last non-null), maximum, minimum, and mean values for all four sensor types evaluated over the entire active session up to the prediction request:

- **Temperature**: `currentTemperature`, `maxTemp`, `minTemp`, `meanTemp`
- **Humidity**: `currentHumidity`, `maxHumidity`, `minHumidity`, `meanHumidity`
- **CO2**: `currentCO2`, `maxCO2`, `minCO2`, `meanCO2`
- **Light**: `currentLight`, `maxLight`, `minLight`, `meanLight`

This complete set of linearized features is passed to the `/predict` API endpoint to evaluate the user's current focus `rating`, providing the ML service with the full spectrum of environmental data.

### 3.6.3 Data Split and Validation Strategy

To ensure robust evaluation and prevent data leakage, we implemented a 64/16/20 split for training, validation, and testing:

1. **Initial Split**: The dataset is split into an 80% development set (training + validation) and a 20% hold-out test set.
2. **Validation Split**: The development set is further split, reserving 20% of it (16% of the total data) for validation and hyperparameter tuning, leaving 64% for model training.

Both splits employ **stratified sampling** based on the target `rating` variable to guarantee that all three subsets maintain the same proportional distribution of user ratings. However, a dynamic fallback is implemented to temporarily disable stratification if extreme class imbalances are present (e.g., if a rating class has fewer than two samples). The final models are evaluated primarily using **Accuracy** and **Macro F1-score**.

Initially ideal test set would be the one taken from the real usage of the system (both iot and frontend) but since not enough data was received this could not have been accomplished.

## 3.7 Frontend Implementation

Authors: [Name, Name]

### 3.7.1 Core Features Implementation

[Describe how the key features were built:

- Fetching, parsing, and displaying live sensor data from the REST API
- Historical data visualisation (charts/graphs — which library: Recharts, Chart.js, D3?)
- User interactions and controls for managing the system
- Any state management patterns worth highlighting]

```
/* Include a representative React component illustrating a key implementation
   decision – e.g. a data-fetching hook, a chart component, or an API call. */
```

### 3.7.2 API Integration

[How does the frontend communicate with the backend? Describe error handling, loading states, polling vs websocket decisions, and how ML predictions are retrieved and displayed.]

### 3.7.3 Hosting and Deployment

[Describe how the React app is built and hosted. Where is it deployed? How is the deployment triggered (manual push, CI/CD pipeline)? Provide the live URL if applicable.]

## 3.8 IoT CI/CD

Authors: [Jakub Maciej Baczek, Name]

### 3.8.1 DevOps Considerations for Embedded Development

Integrating CI/CD into an embedded C project targeting the ATmega2560 microcontroller presents challenges that are fundamentally different from those encountered in typical software projects. The most significant of these is the absence of practical emulation: unlike web or desktop applications, firmware for an AVR microcontroller cannot be meaningfully run on a standard x86 Linux host without significant behavioural divergence. This makes end-to-end automated testing on real hardware largely impractical in a CI context, as it would require physical hardware attached to the runner or a dedicated hardware-in-the-loop setup.

A further challenge is that much of the codebase is tightly coupled to hardware-dependent peripherals — UART, Wi-Fi modules, and similar — whose behaviour cannot be exercised without the target hardware. The cross-compilation toolchain adds additional complexity: building firmware for the ATmega2560 requires the AVR-GCC toolchain and PlatformIO, neither of which are part of a standard CI environment, and both of which must be installed and cached correctly to produce a reproducible build.

Automatic deployment presents a similar problem. In a conventional software project,a CD pipeline can push a build artifact directly to its destination — a server, a container registry, a package repository. For embedded firmware, deployment means physically flashing the binary onto the microcontroller, which cannot be done without direct access to the hardware. Fully automated deployment is therefore not feasible in this context. The practical solution adopted here is to treat the compiled firmware as the deployable artifact: the pipeline produces a flashable `.hex` file and uploadsit as a GitHub Actions artifact on every successful build, ready to be downloaded and flashed to the board manually.

These constraints were acknowledged from the outset, and the CI/CD strategy was designed accordingly. Rather than attempting to run firmware on the target or simulate peripherals fully, the CI side of the pipeline focuses on two concerns: automated unit testing of hardware-independent logic, compiled and run natively on the CI runner using GCC, and a firmware build step using PlatformIO to verify that the codebase compiles correctly for the ATmega2560 target. The CD side is reduced to producing and publishing the `.hex` artifact, deferring the final flashing step to the developer. This separation allowed meaningful automation despite the inherent limitations of embedded CI/CD.

### 3.8.2 Tools and Pipeline

## CI Pipeline

The CI pipeline is implemented using GitHub Actions and is defined in a single workflow file. It is triggered on pull requests targeting the `main` and `dev` branches. To avoid unnecessary work when unrelated parts of the repository change, the pipeline uses `dorny/paths-filter` to check for modifications within the `IOT/` directory and skips all subsequent jobs if none are found.

The pipeline is divided into three sequential jobs:

#### `detect-changes` — Change Detection

Runs on every pull request and uses `dorny/paths-filter` to determine whether any files under `IOT/` were modified. The `iot-test` and `iot-build` jobs are conditional on this check, and are skipped entirely if no relevant changes are detected.

#### `iot-test` — Testing and Coverage

Runs on an `ubuntu-latest` runner and is responsible for compiling and executing the unit test suite natively. Hardware-dependent subsystems — such as UART drivers, SPI communication, and Wi-Fi module interaction — cannot run on a host machine and were therefore isolated behind fakes and stubs using the FFF (Fake Function Framework). These fakes are placed under `test/fakes/` and are included at compile time via the `-I./test/fakes` flag, replacing real peripheral drivers with non-operational or configurable substitutes. This allows the logic within modules such as `wifi_http.c` and `server_api.c` to be tested in isolation without any hardware dependency.

Tests are written using the Unity unit test framework for C, with test files compiled and linked against the source under test and the Unity runner. The `make coverage` target compiles all test binaries with GCC's `--coverage` flag (gcov instrumentation), executes them, and then uses `lcov` and `genhtml` to produce a HTML coverage report. `lcov` is installed via `apt-get` as a pipeline step. Third-party and test infrastructure paths (`fakes/`, `unity/`, `test/`, `/usr/`) are excluded from the coverage data to ensure only production source is measured. The resulting HTML report is uploaded as a GitHub Actions artifact named `coverage-report` for inspection after each run.

The job also requires a `secrets.ini` file containing build flags for credentials such as Wi-Fi SSID and server host. Since these cannot be stored in the repository, the file is generated dynamically in the pipeline using placeholder values sufficient for compilation and testing.

#### `iot-build` — Firmware Compilation

Runs only if `iot-test` succeeds and is responsible for verifying that the firmware compiles correctly for the ATmega2560 target using PlatformIO. PlatformIO is installed via `pip`, with the `~/.platformio` directory cached using `actions/cache` keyed on the hash of `platformio.ini` to avoid redundant downloads across runs. Like `iot-test`, this job also generates a `secrets.ini` with placeholder values before building. The build targets the `megaatmega2560` PlatformIO environment, and the resulting `firmware.hex` file — ready to be flashed to the microcontroller — is uploaded as a build artifact named `firmware`. This provides a verifiable, reproducible binary for every pull request that passes testing.

### 3.8.3 Integration into Workflow

The CI pipeline was integrated directly into the pull request workflow on GitHub. All pull requests targeting `main` or `dev` were required to either pass the `iot-test` and `iot-build` jobs or have no IoT-related changes before merging was permitted — a failing test run or a broken firmware build would block the PR. This ensured that neither regressions in testable logic nor compilation failures could be introduced into the protected branches.

Responsibility for fixing a broken build was straightforward: the author of the pull request that caused the failure was expected to resolve it. This kept accountability clear and avoided a situation where broken builds were left for others to diagnose. In practice, outright compilation failures were rare, as code was manually verified on the Arduino before being pushed. The most common failure mode was instead test failures arising from changes to modules covered by the Unity test suite.

### 3.8.4 Outcomes and Evaluation

The DevOps integration was largely successful within the constraints imposed by embedded development. The two-job pipeline structure — separating native unit testing from cross-compiled firmware verification — proved to be a practical and effective approach. Compilation errors targeting the ATmega2560 were caught automatically on every PR, and the unit tests provided a degree of confidence in the correctness of the hardware-independent application logic.

The use of FFF for faking peripheral dependencies worked well in practice: modules could be tested in isolation without requiring any hardware, and the fake implementations were straightforward to write and maintain. The Unity framework similarly integrated cleanly with the native GCC compilation path and the lcov coverage toolchain.

The most significant limitation of the pipeline is the scope of what could be tested automatically. Because tests run natively on an x86 host, only code that could be cleanly decoupled from hardware-specific behaviour was testable in CI. Driver-level code — responsible for directly interfacing with UART, SPI, or the Wi-Fi module — was excluded from automated testing entirely, as no meaningful substitute for actual hardware execution exists at that level. Full integration testing, including end-to-end verification of sensor readings and server communication over a real network, remained a manual process conducted on physical hardware.

Overall, the pipeline added clear value to the development workflow by catching build and logic errors early, enforcing a baseline standard for all contributions, and producing a deployable firmware artifact on every successful PR. The inability to automate hardware-level testing is an inherent property of the embedded domain rather than a gap in the implementation, and the pipeline was scoped accordingly.

## 3.9 Machine Learning — Models

Authors: [Piotr Junosz, Name]

In our project, we developed two distinct kinds of models to tackle different aspects of the Study Suitability problem:

1. **Session-based Models:** These models rely on chronological session data where linearization is in place. They account for the aspect of environment changes over time.
2. **Instant Measurement Models:** These models rely solely on point-in-time environmental sensor data (temperature, humidity, noise, co2, light) to predict the user's study suitability. We rigorously evaluated multiple approaches for this instant measurement prediction pipeline.

### 3.9.1 Model Selection

1. For the instant measurement predictions, we evaluated both regression and classification approaches since `comfortValue` is an ordinal rating (1 to 5). The models evaluated include:

- **Linear Regression (LR):** Used as a baseline regressor to establish whether linear relationships exist between sensors and comfort.
- **Random Forest Regressor (RFR):** Chosen for its ability to capture complex, non-linear interactions between environmental variables without requiring extensive feature scaling.
- **Random Forest Classifier (RFC):** Treats the 1-5 comfort ratings as distinct classes, aiming to accurately predict the exact category.
- **Gradient Boosting Classifier (GBC):** An advanced ensemble technique evaluated for its ability to sequentially correct prediction errors, offering potentially higher accuracy for the subjective ratings.
- **Multi-Layer Perceptron (MLP):** A feedforward artificial neural network evaluated to see if deep, non-linear pattern recognition could better map raw sensors to human comfort.
- **Two-Stage Pipeline:** A custom hybrid approach where Stage 1 uses Random Forest Classifiers to map raw sensor data into intermediate human "perception" values (e.g., Temperature -> "Cold", "Perfect", "Hot"), and Stage 2 evaluates both a Random Forest Regressor and a Random Forest Classifier on those intermediate perceptions to predict the final 1-5 comfort rating. Those "perception" values already existed in original data that was found together with the comfort raiting, so this experiment was supposed to see if this way models can predict more accurately - it turned out that not really.

### 3.9.2 Training and Hyperparameter Tuning

To guarantee rigorous Data Science methodology, all instant models were trained using a strict **80% Training / 20% Test** split. We eliminated redundant validation splits to maximize training data utility.

Instead of manual tuning, we leveraged `GridSearchCV` with 5-fold cross-validation exclusively on the 80% training set to find optimal hyperparameters. To prevent data leakage, the 20% test set was locked away during the Grid Search and only evaluated once at the very end of the experiment.

The complete parameter grids searched during the tuning phase for each model were:

- **Linear Regression:**
  - `fit_intercept`: [True, False]
- **Random Forest Regressor:**
  - `n_estimators`: [100, 300]

- `max_depth`: [None, 10, 20]
- `min_samples_split`: [2, 5]
- `min_samples_leaf`: [1, 3]

- **Random Forest Classifier:**
  - `n_estimators`: [100, 300]
  - `max_depth`: [None, 10, 20]
  - `min_samples_split`: [2, 5]
  - `min_samples_leaf`: [1, 3]
- **Gradient Boosting Classifier:**
  - `n_estimators`: [50, 100, 200]
  - `learning_rate`: [0.05, 0.1]
  - `max_depth`: [3, 5]
- **Multi-Layer Perceptron:**
  - `hidden_layer_sizes`: [(8,), (16,)]
  - `alpha`: [0.0001, 0.01]
  - `learning_rate_init`: [0.001, 0.005]
- **Two-Stage Pipeline (Stage 2 Regressor):**
  - `n_estimators`: [100, 200]
  - `max_depth`: [None, 10, 20]
- **Two-Stage Pipeline (Stage 2 Classifier):**
  - `n_estimators`: [100, 200]
  - `max_depth`: [None, 10, 20]
  - `class_weight`: ['balanced', None]

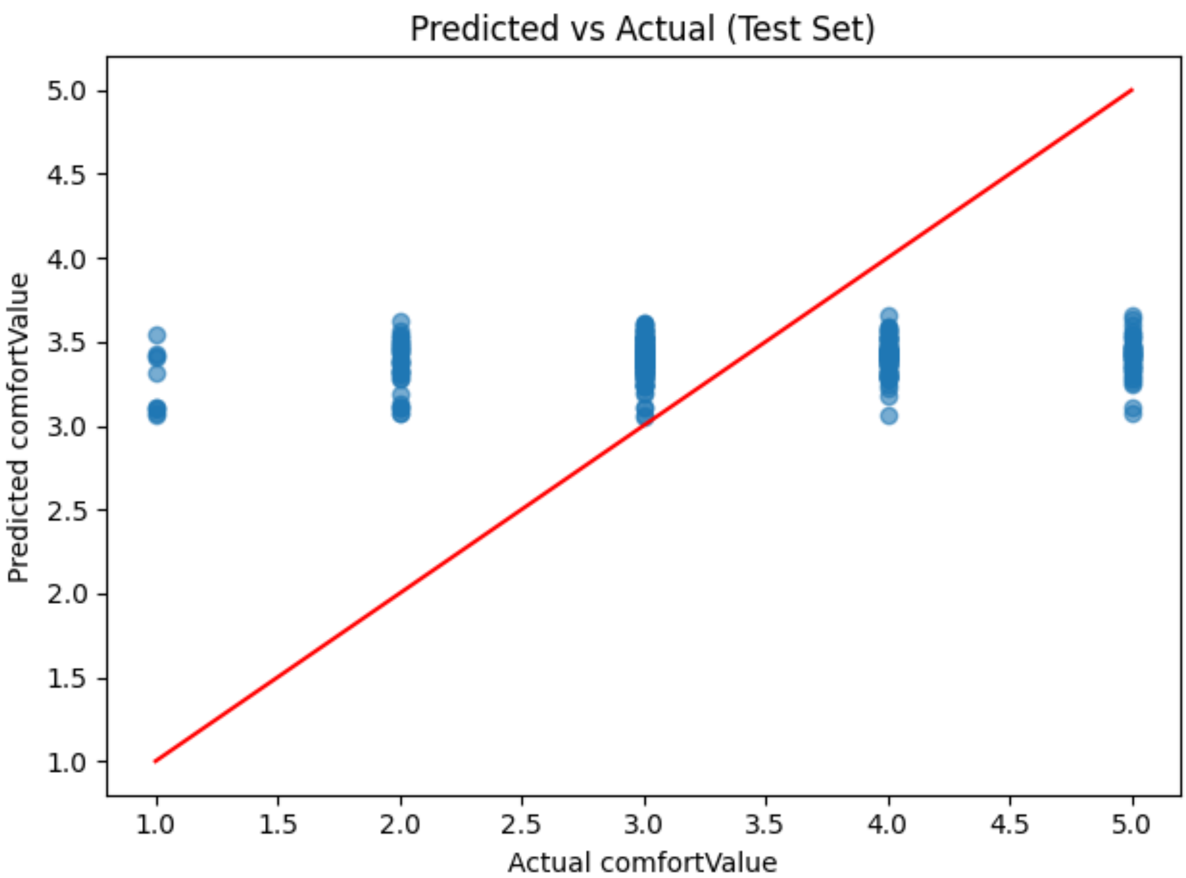
### 3.9.3 Model Evaluation

The models were evaluated strictly on the holdout test set to determine how well point-in-time sensor data correlates to a user's comfort. Regressors were evaluated using Mean Absolute Error (MAE), while classifiers were evaluated on Accuracy.

| Model | Type | Test Metric | Result |
|---|---|---|---|
| Linear Regression | Regressor | MAE | 0.749 |
| Random Forest Regressor | Regressor | MAE | 0.737 |
| Random Forest Classifier | Classifier | Accuracy | 37.2% |

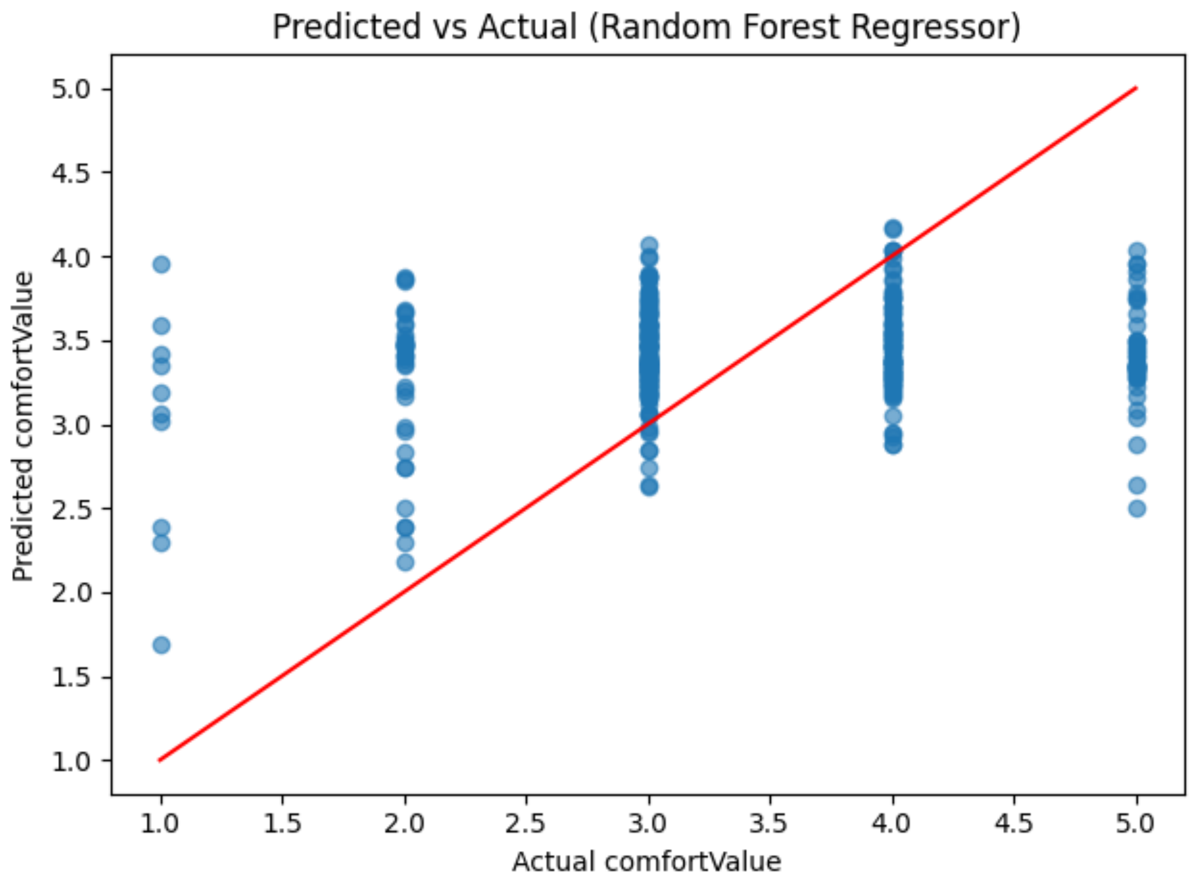
| Model | Type | Test Metric | Result |
|---|---|---|---|
| Gradient Boosting Classifier | Classifier | Accuracy | 40.7% |
| Multi-Layer Perceptron | Classifier | Accuracy | 46.1% |
| Two-Stage Pipeline | Hybrid | MAE / Acc | 0.667 / 48.5% |

**Linear Regression**



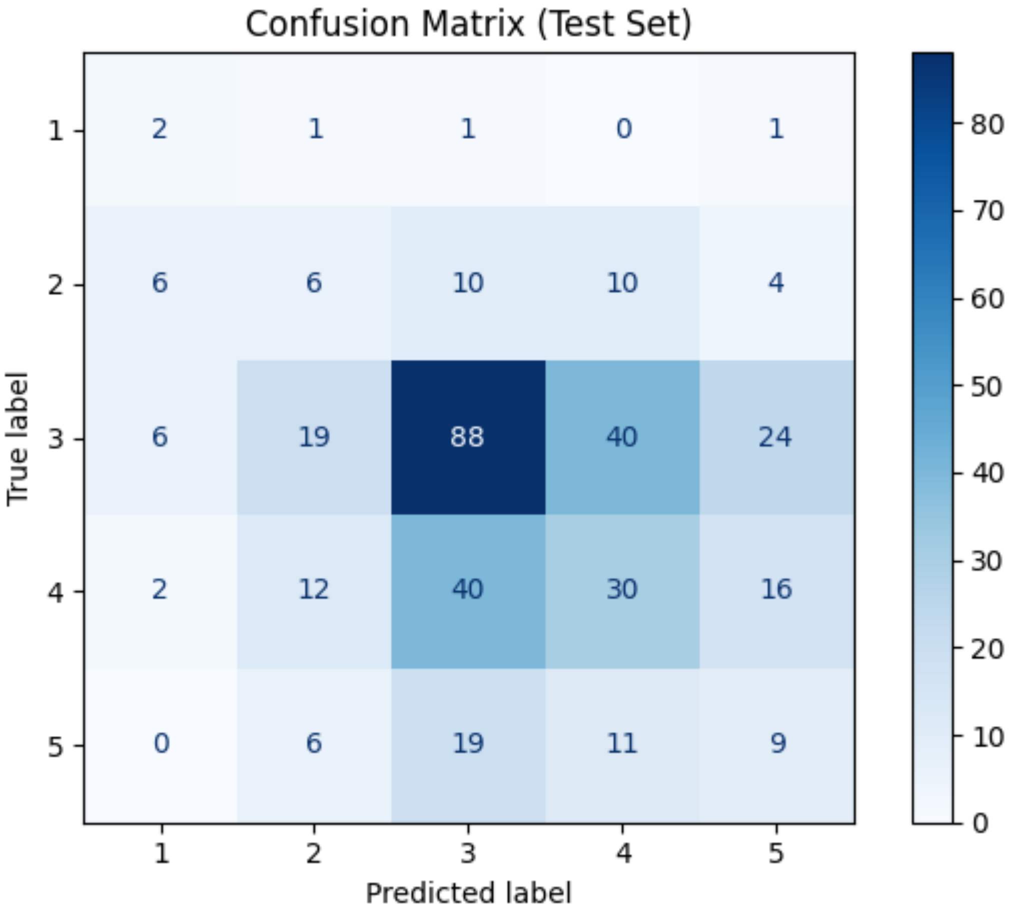

In the basic linear regression model the predicted values were all around the most common value (3) which suggested that the problem might need more complex model than linear.

**Random Forest Regressor**



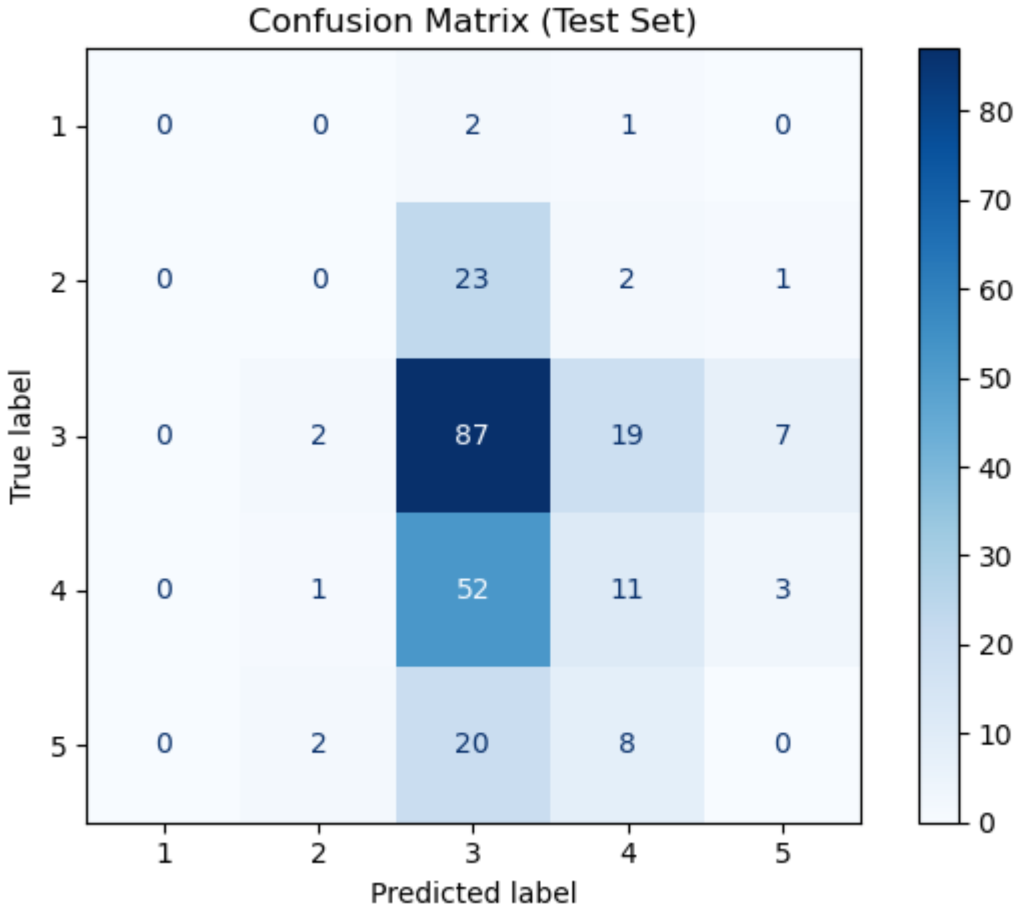

Random forest regressor did a little better - it preserved more variance and the predicted values where compressed in the middle most common value but not as much as in the linear regression, due to [continue].

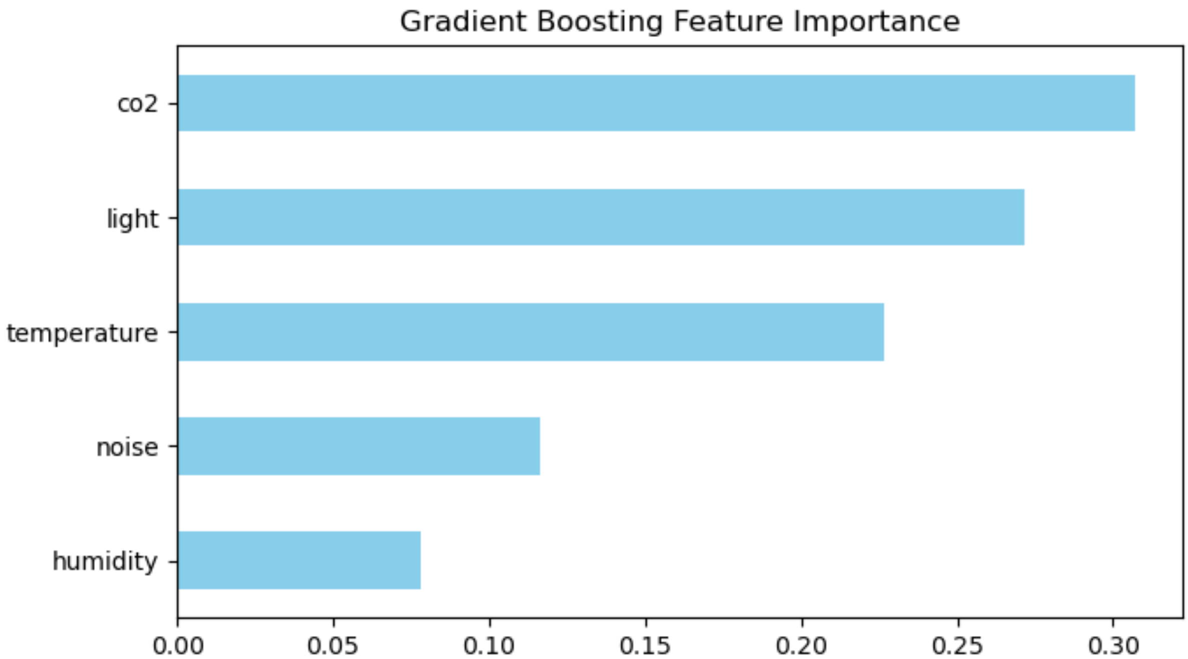
**Random Forest Classifier**

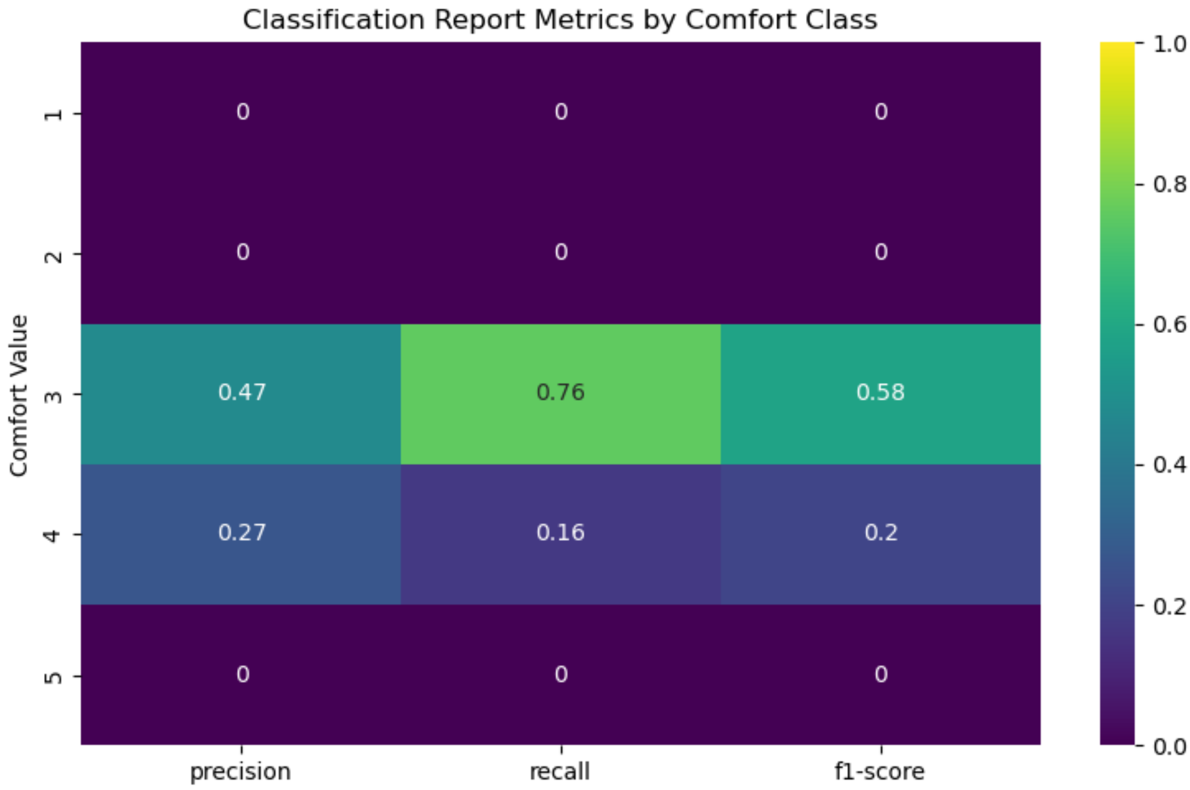



First classification model experiments were done on Random Forest Classifier and showed that classification handles this problem slightly better preserving even more variance of predicted values where also extreme classes were included sometimes.

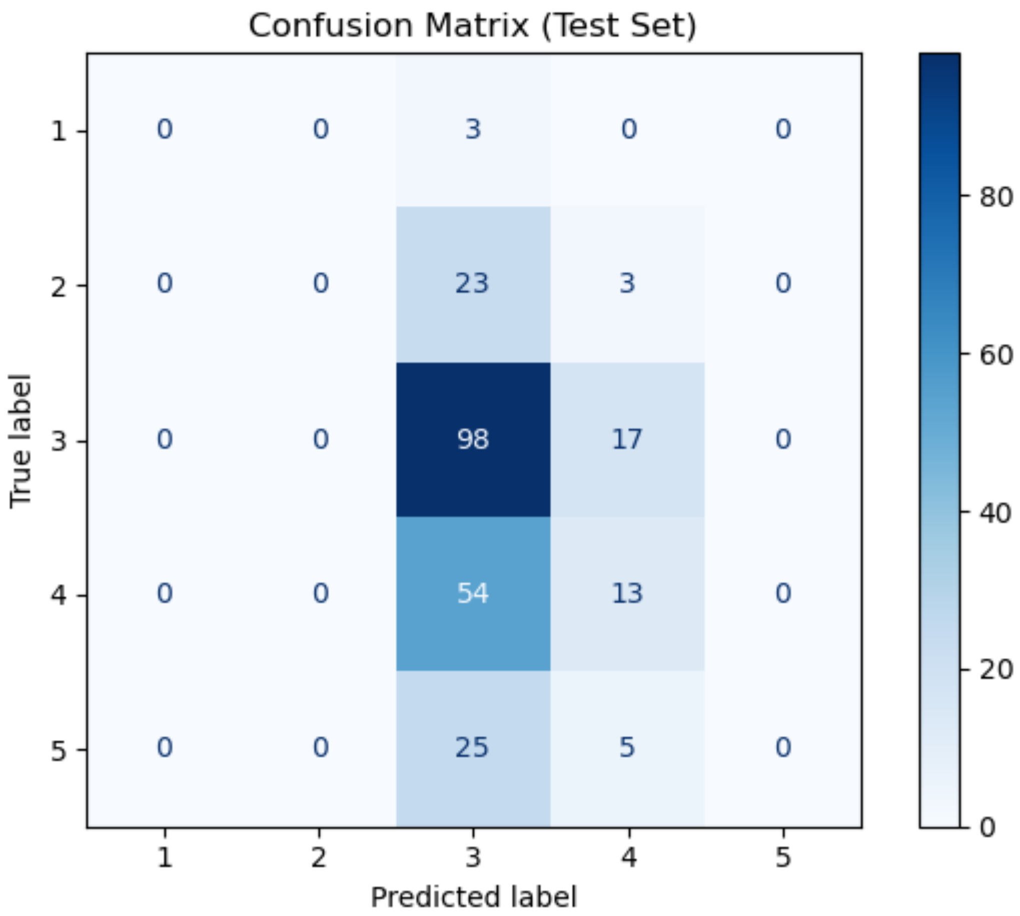
**Gradient Boosting Classifier**

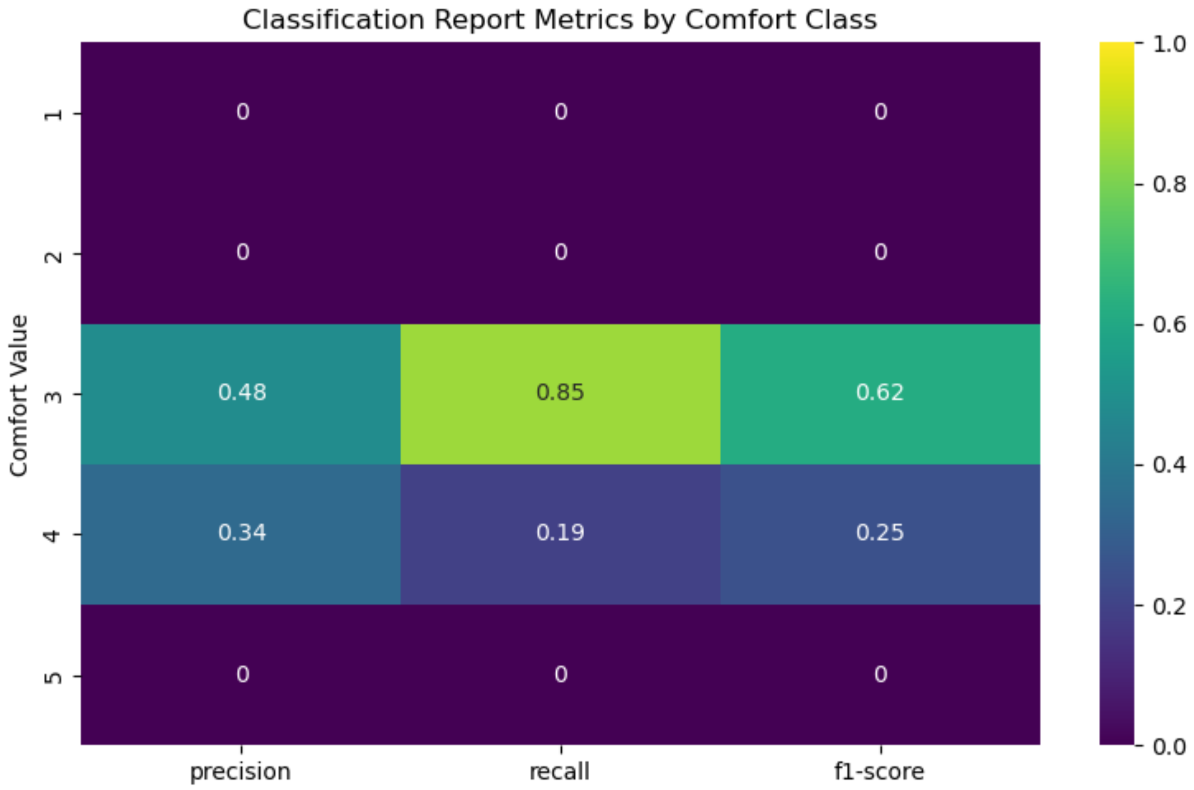

Confusion Matrix (Test Set)
True label
Predicted label
1
2
3
4
5
0
0
2
1
0
0
0
23
2
1
0
2
87
19
7
0
1
52
11
3
0
2
20
8
0
80
70
60
50
40
30
20
10
0

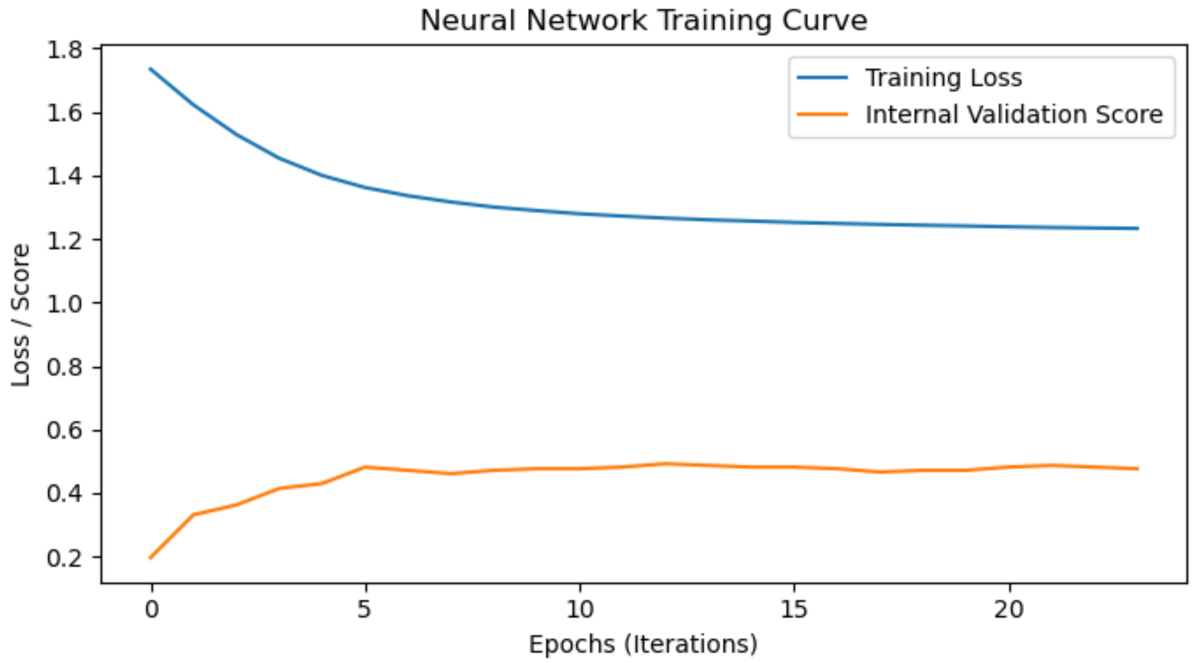

Gradient Boosting Feature Importance
co2
light
temperature
noise
humidity
0.00
0.05
0.10
0.15
0.20
0.25
0.30



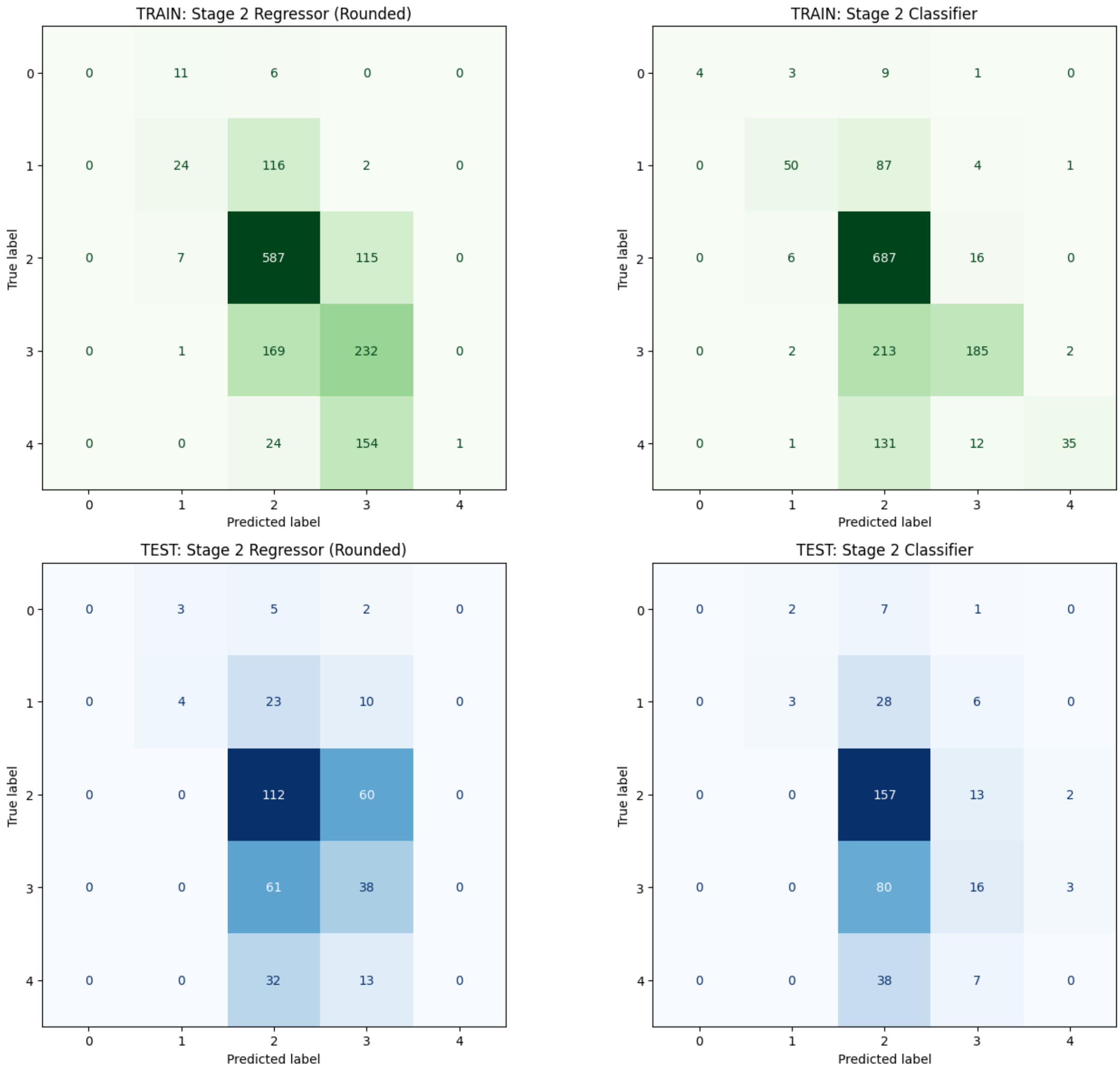

**Multi-Layer Perceptron**


Confusion Matrix (Test Set)
True label
Predicted label
1
2
3
4
5
0
0
3
0
0
0
0
23
3
0
0
0
98
17
0
0
0
54
13
0
0
0
25
5
0
80
60
40
20
0




**Two-Stage Pipeline**




## Critical Evaluation: The “Subjectivity Paradox”

As clearly visible in the confusion matrices, the instant measurement models struggled to capture a robust predictive signal. The models typically exhibited two failure modes: they either heavily overfitted to the training data, or they collapsed into predicting the majority class.

Initially, this lack of predictive power was viewed as a failure of the machine learning pipeline, leading to frustration regarding the feasibility of the feature. However, upon deeper analysis, this outcome is actually one of the most valuable findings of the project. It empirically proves what we term the **“Subjectivity Paradox”**: raw physical sensor parameters (temperature, humidity, noise, etc.) are inherently insufficient to objectively predict human comfort. Two different users in the exact same room with identical environmental readings can give vastly different comfort ratings.

A universal instant-comfort model cannot exist. To solve this, future iterations of the system must rely on personalized, user-specific profiling or hardcoded rules per user rather than attempting to map objective sensor data to a generalized subjective comfort scale.

### 3.9.4 Result ExportThe best performing models were serialized into `.pkl` files (Pickle format). This allows the backend API to dynamically load the model into memory and instantly run predictions when receiving new raw sensor arrays from the physical IoT devices.

## 3.10 Frontend CI/CD

Authors: [Name, Name]

### 3.10.1 DevOps Considerations for the Frontend

[What DevOps planning was done specifically for the React codebase? How were code ownership and review responsibilities organised?]

### 3.10.2 Tools and Pipeline

[Describe the CI/CD pipeline for the frontend:

- Linting and formatting (ESLint, Prettier)
- Unit and component tests (Jest, React Testing Library)
- E2E tests if applicable (Cypress, Playwright)
- Automated build and deployment to hosting
- Container usage for frontend (if applicable)]

### 3.10.3 Integration into Workflow

[How was the pipeline used day-to-day? Branch protection rules, required checks?]

### 3.10.4 Outcomes and Evaluation

[What effect did DevOps have on frontend quality and development speed? What automated checks ran on every PR? What gaps remained?]

## 3.11 IoT Tests

Authors: [Jakub Maciej Baczek, Name]

### 3.11.1 Testing Strategy for Embedded C

Unit testing for the IoT application focused on three modules containing application logic: wifi_http, responsible for establishing TCP connections and constructing HTTP requests; server_api, responsible for session management and server communication; and display_status, responsible for driving the four-digit segment display with status-specific patterns. Tests were written using the Unity unit test framework for C and compiled natively on the host using GCC, allowing them to run in CI without any physical hardware.

Hardware-dependent components — TCP socket operations, Wi-Fi driver commands, buzzer output, and display driver calls — were replaced with fakes generated using the FFF (Fake Function Framework). FFF allows individual driver functions to be substituted with configurable stubs that record call counts, capture arguments, and inject return values or response payloads. This made it possible to test application logic such as session ID parsing, retry behaviour, endpoint construction, and display output in full isolation from the underlying hardware.

The remaining codebase consists of low-level peripheral drivers and the main application loop. Drivers are inherently hardware-dependent and cannot be meaningfully tested without the target device. The main loop contains no isolated logic of its own, acting purely as an orchestrator of the other modules. Neither lends itself to unit testing, and both were verified manually on the physical hardware.

### 3.11.2 Unit Test Results

| Module | Tests | Passed | Failed | Coverage |
|---|---|---|---|---|
| `wifi_http` | 18 | 18 | 0 | 93.9% |

| Module | Tests | Passed | Failed | Coverage |
|---|---|---|---|---|
| `server_api` | 36 | 36 | 0 | 96.6% |
| `display_status` | 11 | 11 | 0 | 100% |

### 3.11.3 Integration and System-Level Tests

No automated integration testing was implemented, as the hardware constraints discussed in section 3.8.1 make this impractical without a dedicated hardware-in-the-loop setup. Integration and system-level verification was instead performed manually throughout development. This involved flashing the firmware onto the ATmega2560 and observing end-to-end behaviour: confirming that sensor readings were correctly acquired, formatted into the expected JSON payloads, transmitted to the server, and that session lifecycle events such as session start and pulse updates were handled correctly. While informal, this process provided confidence in the integrated behaviour of the system and complemented the unit-level coverage achieved through automated testing.

## 3.12 Frontend Tests

Authors: [Name, Name]

### 3.12.1 Testing Strategy

[What test types were applied? Unit tests (component rendering), integration tests (API mocking), or E2E tests (full browser automation)?]

### 3.12.2 Test Results

| Test Suite | Tests | Passed | Failed | Coverage |
|---|---|---|---|---|
| Component tests | | | | |
| Integration tests | | | | |

### 3.12.3 Responsiveness Testing

[How was responsive behaviour verified at the three required breakpoints (576px, 768px, 1200px)? Include screenshots if helpful.]

## 3.13 Machine Learning Tests and DevOps (MLOps)

Authors: [Piotr, Name]

### 3.13.1 Machine Learning Testing Strategy

The Machine Learning and API (MAL) component is verified through a multi-layered testing strategy that ensures both the data processing logic and the serving infrastructure are robust.

**Data Pipeline Testing** We implemented unit and integration tests for the data transformation logic (e.g., `test_build_unified_environment_dataset.py`). These tests verify:

- **Merging Logic**: Ensuring that disparate datasets (IoT sensor logs, study ratings, and environmental history) are correctly joined on time-series keys.
- **Preprocessing Correctness**: Validating that the MICE imputation and k-means clustering logic produce consistent outputs without introducing data leakage.
- **Schema Validation**: Ensuring the final processed dataset matches the input requirements of the Random Forest model.

**API and Model Serving Testing** To verify the serving layer, we use `pytest` (e.g., `test_prediction_api.py`) to test the FastAPI endpoints. These tests serve as integration checks that:

- **Endpoint Availability**: Confirm the `/predict` and health check endpoints respond correctly.
- **Model Loading**: Verify the `rf_model.pkl` artifact is correctly loaded and can produce predictions.
- **Input Validation**: Test the API's resilience to malformed or out-of-range sensor data.
- **Inference Correctness**: Validate that the prediction output matches the expected schema and logical bounds of the Study Suitability Rating.

### 3.13.2 MLOps Considerations

The MAL component requires a specialized DevOps approach to manage the lifecycle of both code and serialized model weights. The primary challenge is ensuring that changes to the data processing logic in `ml_pipeline/` are always compatible with the model artifact committed in `models/`. Unlike traditional software, the "build" artifact in MLOps includes both the code and the serialized model weights (`rf_model.pkl`).

### 3.13.3 Tools and Pipeline

The MLOps pipeline is automated via GitHub Actions (`mlops.yaml`) and executes the testing strategy described above on every pull request.

#### `test-and-train` [TODO: maybe change the name because does not inlcude "train"]Job

The pipeline automates several critical verification steps:

1. **Environment Setup**: Python 3.10 is configured with dependencies, using a PostgreSQL sidecar for realistic data integration checks.
2. **Automated Verification**: The pipeline runs the full `pytest` suite, including the data pipeline and API tests. This ensures that no code change breaks the existing model's ability to serve predictions.
3. **Model Artifact Integrity**: The workflow explicitly fails if the `rf_model.pkl` is missing or corrupted, preventing "empty" deployments.
4. **Containerized Continuous Delivery**: On successful validation and merge to `main`, the pipeline builds a Docker image (`mal-api`) and pushes it to the GitHub Container Registry (GHCR).

### 3.13.4 Outcomes and Evaluation

This integrated Testing and MLOps approach significantly reduced deployment risks. By coupling data transformation tests with live API integration checks, we ensured that the entire pipeline—from raw data to study suitability prediction—is verifiable and reproducible. The use of Docker images for deployment ensures that the exact environment used during CI is replicated in production.

# 4. RESULTS AND DISCUSSION

Authors: [Name, Name, Name]

## 4.1 Integrated System Results

[How does the complete system perform end-to-end? Demonstrate all three parts working together (reference the submitted demo video). What does actual sensor data look like flowing through to the frontend predictions?]

## 4.2 Evaluation Against Objectives

[Revisit each objective from Section 1.3. For each, state whether it was met, partially met, or not met, and support the assessment with evidence.]

| Objective | Status | Evidence |
|---|---|---|
| **IoT** — measure and transmit sensor readings every ≤60 s | ✔ Met | §3.2.1 sensor hardware; §3.2.2 30 s data / 5 s pulse timers; §3.5.1 driver implementation and $CO_2$ fallback caching; §3.5.2 HTTP transmission |
| **Cloud Backend** — persist sensor data and expose a RESTful API | ✔ Met | §3.1.1 Core API architecture; §3.1.2 Docker Compose and schema init; §2.3 FR02–FR03; §4.6 stack stable throughout project period |
| **Machine Learning** — train a 1–5 suitability classifier and expose via API | ✔ Met | §3.3.3 multi-class classification formulation; §3.6.2 16-feature session vector; §3.9.1–§3.9.3 model selection, tuning, and evaluation; §3.13.1 `/predict` endpoint verified |

| Objective | Status | Evidence |
|---|---|---|
| **Frontend** — display live readings and ML rating responsively | ✔ Met | §3.4.1 UI/UX design; §3.4.3 breakpoints at 576 px, 768 px, 1200 px; §3.7.1 data fetching and chart implementation; §3.12.3 responsiveness testing; §2.3 FR05 |
| **DevOps** — containerise all components and enforce CI/CD pipelines | ✔ Met | §3.1.2 all services in `docker-compose.yml`; §3.8.2 `iot-test` and `iot-build` jobs; §3.13.3 MLOps pipeline and GHCR publish; §4.6 zero manual deployment effort |
| **Security** — encrypt IoT-to-backend; protect frontend API endpoints | ⟳ Partial | §3.1.3 JWT + bcrypt for frontend endpoints; IoT-to-backend remains plain HTTP; §2.3 NFR01 not fully satisfied; secret management via environment variables enforced in `docker-compose.yml` |

## 4.3 IoT Performance

Sensor reads proved reliable throughout testing, with no data loss or transmission failures observed. The DHT11 is read immediately before each transmission, so temperature and humidity values are always current. The $CO_2$ sensor follows a request-then-read pattern across cycles due to its internal measurement delay; if a fresh reading is unavailable, the system falls back to the last cached value and continues transmitting uninterrupted.

All buffers are statically allocated at compile time, avoiding heap fragmentation on the ATmega's limited SRAM. Each HTTP request opens a TCP connection, waits up to 3 seconds for a response with watchdog resets in the busy-wait loop, then closes — keeping memory usage predictable and preventing watchdog-triggered reboots during legitimate network waits. No latency issues, sampling drift, or memory problems were observed during operation.

## 4.4 ML Performance

[Summarise the final model performance in context. Are predictions accurate enough to be useful for the application? How does the model compare to a naive baseline?]

## 4.5 Frontend Quality

[Does the application meet the functional requirements? Does it display data correctly across all required breakpoints? Are all customer-facing features accessible via the UI?]

## 4.6 Cloud and DevOps Evaluation

The system has no serverless workloads; all services run as containers managed by Docker Compose. Under normal use — a single device sending sensor readings every 30 seconds — the stack performed without issues throughout the project period. The PostgreSQL database, main API, ML API, and frontend all started cleanly in the correct order, served requests as intended, and retained data across restarts. The ML API consistently returned study quality predictions, and the frontend remained available throughout testing. The system was also briefly tested with three concurrent devices and performed without issues.

Deployment to Coolify triggers automatically on every push to `main`, with a concurrency lock preventing overlapping deployments. CI pipelines run path-filtered tests for the API, IoT firmware, and ML service on every pull request, meaning only relevant workflows run and feedback stays fast. Only `dev` or `fix/*` branches may be merged into `main`, enforcing a consistent workflow throughout the project. Overall, the DevOps setup reduced manual deployment effort to zero and caught regressions early.

## 4.7 Critical Evaluation and Limitations

[Honestly evaluate validity and reliability of your results. What are the system's remaining weaknesses? What assumptions constrain the findings? What would need to change for this to be a production-grade system? Address limitations per component where the issues differ significantly.]

# 5. CONCLUSIONS

Authors: [Name, Name, Name]

[Summarise the project in 3–5 paragraphs:

1. Restate the problem and the three-part approach taken
2. What was achieved — per component and as an integrated system
3. Did the system solve the stated problem, and to what degree?
4. What is the overall answer to the problem statement?]

# 6. FUTURE WORK

Authors: [Name, Name, Name]

[Describe the most important directions for continuing or improving the system. Be specific and actionable.

- **IoT**: additional sensors, power optimisation, OTA firmware updates, PCB design…
- **ML**: more training data, online/incremental learning, alternative models, explainability…
- **Frontend**: mobile app, push notifications, user authentication, dashboard customisation…
- **Cloud/DevOps**: full CD with staged environments, infrastructure-as-code (Terraform), monitoring and alerting (Grafana, Prometheus), load testing…]

# REFERENCES

# APPENDICES

## Appendix A — Source Code

| Component | Repository URL |
|---|---|
| IoT | [GitHub/GitLab URL] |
| Cloud | [GitHub/GitLab URL] |
| ML | [GitHub/GitLab URL] |
| Frontend | [GitHub/GitLab URL] |

## Appendix B — API Documentation

[Link to hosted API docs (Swagger/OpenAPI), or include the endpoint specification here.]

## Appendix C — Hardware Schematics

## Appendix D — Additional ML Figures